SoftCreate POS

EDICIÓN TEMÁTICA HALLOWEEN • ARCHITECTURE BLUEPRINT

MANUAL TÉCNICO OFICIAL

DOCUMENTACIÓN INTEGRAL DE INGENIERÍA DE SOFTWARE

POS HALLOWEEN SISTEMA DE PUNTO DE VENTA

Auditoría integral de código, arquitectura de componentes bajo Atomic Design, catálogo descriptivo archivo por archivo, guía de integración para desarrolladores y orquestación de hardware térmico en la nube.

FRONTEND STACK	ESTADO & SERVIDOR	BACKEND & BD	HARDWARE & PRINTING
React 19 + Vite 7	**Zustand + TanStack Query**	**Supabase Cloud + PostgreSQL**	**QZ-Tray + ESC/POS + pdfmake**

Confidencialidad: Documentación Interna para Desarrolladores

Fecha de Generación: Octubre 2026

Versión: 1.0.0 Enterprise

Tabla de Contenidos

CAPÍTULO 1: ESTRUCTURA GENERAL DEL SISTEMA Y ARQUITECTURA

- 1.1 Propósito, Alcance y Dominio del Sistema POSHalloween
- 1.2 Stack Tecnológico Principal y Matriz de Dependencias
- 1.3 Paradigma Arquitectónico en 3 Capas de Frontend
- 1.4 Metodología de Diseño UI: Atomic Design Jerárquico
- 1.5 Modelo de Seguridad y Control de Acceso Basado en Roles (RBAC)
- 1.6 Mapa Jerárquico de Directorios y Estructura Raíz

CAPÍTULO 2: DESGLOSE DETALLADO ARCHIVO POR ARCHIVO

- 2.1 Archivos Raíz, Entorno y Configuración del Ecosistema
- 2.2 Núcleo de Entrada y Sistema de Enrutamiento (/src, /routers)
- 2.3 Capa de Contextos y Hooks Personalizados (/context, /hooks)
- 2.4 Capa de Estado Global del Cliente con Zustand (/store - 32 stores)
- 2.5 Capa de Integración y Procedimientos Almacenados Supabase (/supabase - 27 archivos)
- 2.6 Capa de Sincronización y Mutaciones TanStack Query (/tanstack - 24 módulos)
- 2.7 Capa de Vistas y Páginas de Navegación (/pages - 22 páginas)
- 2.8 Capa de Componentes UI: Átomos, Moléculas, Organismos y Templates
- 2.9 Reportes de Impresión Térmica, Utilidades y Sistema de Diseño (/reports, /utils, /styles)
- 2.10 Funciones Serverless en el Edge de Supabase (Deno Edge Functions)

CAPÍTULO 3: GUÍA PARA PROGRAMADORES: CÓDIGO CLAVE Y CÓMO USARLO

- 3.1 Flujo de Autenticación, Compuerta de Sesión y Protección Dinámica RBAC
- 3.2 El Ciclo de Vida de una Venta en el POS (Carrito, Zustand y Mutación Atómica)
- 3.3 Orquestación de Hardware Térmico y Apertura de Cajón con QZ-Tray
- 3.4 Motor de Renderizado de Tickets Térmicos en PDF con pdfmake
- 3.5 Control de Arqueo de Caja (Apertura Obligatoria, Conteo Ciego y Cierre)
- 3.6 Trazabilidad Multialmacén y Sistema de Alertas por Stock Mínimo
- 3.7 Guía Paso a Paso para Desarrolladores: Cómo Añadir un Nuevo Módulo al Sistema

CAPÍTULO 4: RESUMEN GENERAL, CONCLUSIONES Y RECOMENDACIONES

- 4.1 Diagnóstico de Madurez y Fortalezas de la Arquitectura

4.2 Oportunidades Críticas de Refactorización y Mejora Técnica

4.3 Flujo de Despliegue en Producción (CI/CD, Firebase, Supabase)

4.4 Resumen de Convenciones y Recomendaciones de Mantenimiento

CAPÍTULO 1

Estructura General del Sistema y Arquitectura de Software

Visión holística del sistema POS Halloween (SoftCreate POS): especificación de alcance comercial, matriz de herramientas tecnológicas, patrones arquitectónicos implementados, modelo de datos y topología de directorios.

1.1 Propósito, Alcance y Dominio del Sistema POSHalloween

POS Halloween (desarrollado bajo la suite SoftCreate POS) es una plataforma de software de Punto de Venta (Point of Sale) de alta concurrencia diseñada para operar en entornos comerciales de comercio minorista y mayorista. Su arquitectura técnica combina la agilidad de una SPA (Single Page Application) moderna con la robustez transaccional de una base de datos relacional PostgreSQL provista por **Supabase**.

El dominio funcional de la aplicación aborda el ciclo integral del comercio físico y digital mediante los siguientes submódulos empresariales:

- **Terminal Punto de Venta (POS):** Interfaz de facturación táctil y de escritorio con búsqueda instantánea de productos por código de barras o descripción, gestión de múltiples almacenes, venta directa, cotizaciones y retención de venta.
- **Control de Turnos y Arqueos de Caja:** Bloqueo operativo hasta la apertura declarada de caja, conteos ciegos de efectivo al finalizar el turno, registro de entradas y salidas manuales de efectivo, y cálculo exacto de sobrantes y faltantes.
- **Integración con Hardware Físico:** Enlace nativo mediante WebSockets firmado digitalmente (QZ-Tray) para comunicarse con impresoras térmicas de 58mm y 80mm, y disparo de pulsos eléctricos en código hexadecimal (ESC/POS) sobre pines RJ11/RJ12 para la apertura de gavetas portamonedas.
- **Gestión de Inventario Multi-Almacén:** Stock segmentado por sucursales y bodegas físicas, transferencias internas, kardex de movimientos de entrada/salida y validación de alertas de stock mínimo tras cada cierre de ticket.
- **Gestión de Créditos y Cuentas por Cobrar:** Ventas a crédito vinculadas a clientes registrados, cálculo de saldos pendientes, registro de abonos parciales, generación de comprobantes de abono y liquidación total.
- **Facturación Automatizada en la Nube:** Función Edge serverless programada en Deno que procesa empresas suscritas, genera facturas PDF mediante `pdf-lib` y las distribuye vía correo electrónico utilizando la API de `Resend`.
- **Theming Especializado:** Personalización estética completa con temática animada de Halloween (sombras violetas, contrastes naranja calabaza, animaciones vectoriales de murciélagos, arañas colgantes y calabazas dinámicas vía Lottie).

> 💡 **Aislamiento y Multi-Tenant por Empresa**
>
> El sistema está diseñado de manera multi-inquilino a nivel de empresa. Todas las consultas transaccionales, almacenes, inventarios, serializaciones y usuarios están indexados y acotados por el identificador único `id_empresa`, garantizando aislamiento de datos estricto.

1.2 Stack Tecnológico Principal y Matriz de Dependencias

El ecosistema tecnológico ha sido seleccionado para proporcionar tiempos de renderizado inferiores a 16 milisegundos por frame y una reactividad instantánea en pantalla de venta:

Categoría	Tecnología / Biblioteca	Versión	Rol en el Sistema
Core UI	React	19.2.0	Librería declarativa de renderizado reactivo con concurrent features.
Empaquetador	Vite	7.3.1	Bundler de ultra-alta velocidad con Hot Module Replacement (HMR).
Enrutador	React Router DOM	7.13.1	Enrutamiento del lado del cliente con control dinámico de rutas protegidas.
Estado Cliente	Zustand	5.0.11	Gestor de estado síncrono ligero con persistencia local en carrito de compras.
Estado Servidor	TanStack React Query	5.90.21	Caché de red, invalidación de consultas, reintentos y deduplicación de peticiones.
BaaS & Backend	Supabase Client JS	2.98.0	Comunicación WebSocket en tiempo real y llamadas RPC a PostgreSQL.
Estilos CSS-in-JS	Styled-Components	6.3.11	Encapsulamiento de estilos mediante tagged template literals y tokens dinámicos.
Animaciones	Framer Motion + Lottie	13.4.0 / 1.2.10	Transiciones suaves entre rutas, micro-interacciones y vectores interactivos.
Hardware Térmico	QZ-Tray + jsrsasign	2.3.0 / 11.1.5	Conexión WebSocket local firmada criptográficamente con algoritmo SHA-512 RSA.
Motor de PDF	pdfmake + print-js	0.2.15 / 1.6.0	Generación client-side de tickets en formato térmico (58mm/80mm) con fuentes embebidas.
Notificaciones	Sonner + SweetAlert2	2.0.7 / 11.26.22	Alertas modales de stock mínimo, advertencias financieras y toasts no intrusivos.

1.3 Paradigma Arquitectónico en 3 Capas de Frontend

A fin de evitar el acoplamiento entre la lógica visual y el acceso a base de datos, POS Halloween implementa una **arquitectura de 3 capas desacopladas**:



Beneficio Técnico: Ningún componente visual ejecuta directamente `supabase.from()` ni gestiona promesas HTTP crudas. Todo flujo visual solicita una mutación a través de TanStack Query, la cual delega la ejecución al módulo CRUD de Supabase, actualiza el estado local de Zustand e invalida las claves de consulta pertinentes para repintar la UI automáticamente.

1.4 Metodología de Diseño UI: Atomic Design Jerárquico

La estructura de `src/components/` respeta estrictamente la metodología **Atomic Design** de Brad Frost, lo que permite la reutilización extensiva de elementos y asegura la coherencia visual del sistema:

- **Átomos (`src/components/atomos/`):** Componentes indivisibles sin lógica de negocio propia (ej. `Linea.jsx`, `Icono.jsx`, `Titles.jsx`, pulsador de apertura de caja).
- **Moléculas (`src/components/moleculas/`):** Combinaciones de dos o más átomos con interacción simple (ej. `Btn1.jsx`, `Spinner1.jsx`, `IndicadorConexion.jsx`, `HistorialCreditoModal.jsx`).
- **Organismos (`src/components/organismos/`):** Secciones complejas de interfaz que encapsulan reglas de negocio y consumen datos (ej. `POSDesing/AreaTecladoPos.jsx`, `formularios/RegistrarProductos.jsx`, `tablas/TablaProductos.jsx`).
- **Templates (`src/components/templates/`):** Disposiciones estructurales y layouts completos de página (ej. `POSTemplate.jsx`, `DashboardTemplate.jsx`, `ConfiguracionesTemplate.jsx`).
- **Páginas (`src/pages/`):** Envoltorios asociados a las rutas de navegación que inyectan los proveedores de datos, validan permisos y renderizan los templates (ej. `POS.jsx`, `Dashboard.jsx`, `Inventario.jsx`).

1.5 Modelo de Seguridad y Control de Acceso Basado en Roles (RBAC)

El control de acceso en POS Halloween no se limita a un rol estático tipo 'admin/cajero'. Posee un sistema de **RBAC dinámico** granular alimentado desde la base de datos:

1. **Compuerta de Autenticación (`AuthGate.jsx`):** Intercepta la carga inicial de la aplicación. Mientras Supabase valida el token de sesión (JWT) en el cliente, el renderizado de la UI se suspende mostrando un spinner temático para evitar "flashes" de pantallas protegidas o redirecciones prematuras.
2. **Verificación de Rutas (`ProtectedRoute.jsx`):** Compara la ruta solicitada en el navegador (`location.pathname`) con la lista de permisos globales asignados al usuario en la base de datos (`item.modulos?.link === location.pathname`). Si el usuario carece del permiso, se desvía a la vista `/sin-permiso`.
3. **Permisos Operativos a Nivel de Botón (`UseValidarPermisosOpertivos.jsx`):** Valida acciones individuales como *"Cobrar venta"*, *"Aplicar descuento"*, *"Abrir cajón sin venta"* o *"Eliminar ítem del carrito"* mediante ventanas modales que exigen privilegios explícitos.

1.6 Mapa Jerárquico de Directorios y Estructura Raíz

A continuación se presenta la cartografía completa del árbol de directorios del proyecto POSHalloween:

Estructura del Proyecto • Árbol de Directorios DIR TREE

```
POSHalloweeen/
├── .env                                # Credenciales de entorno (VITE_SUPABASE_URL, Keys)
├── .firebaserc                         # Configuración de proyectos en Firebase Hosting
├── firebase.json                       # Reglas de enrutamiento SPA y hosting para Firebase
├── vite.config.js                      # Configuración de compilación con Vite y plugins React
├── package.json                        # Manifiesto de paquetes y scripts (dev, build, lint)
├── index.html                          # Punto de entrada HTML5 del bundle
├── public/                             # Recursos estáticos de distribución (logos, fuentes TTF)
├── supabase/                           # Configuraciones de Backend y Edge Functions
│   ├── config.toml                     # Configuración de servicios locales Supabase CLI
│   └── functions/enviar-facturas/      # Microservicio Deno para emisión de facturas PDF por email
└── src/                                # Código fuente principal de la aplicación React
    ├── main.jsx                        # Montaje del Virtual DOM y QueryClientProvider
    ├── App.jsx                         # Orquestación de Providers, Tema Halloween y AuthGate
    ├── index.js                        # Barrel exportador universal (>250 módulos expuestos)
    ├── assets/                         # Archivos multimedia, imágenes e ilustraciones Lottie JSON
    ├── components/                     # Componentes bajo Metodología Atomic Design
    │   ├── atomos/                     # Átomos básicos de UI (botones, iconos, títulos, separadore
s)
    │   ├── moleculas/                  # Moléculas interactivas (buscadores, toggles, modales)
    │   ├── organismos/                 # Módulos complejos de negocio organizados por dominio:
    │   │   ├── AlmacenesDesign/        # Vistas de almacenes físicos
    │   │   ├── DashboardDesign/        # Tarjetas métricas y gráficos Recharts
    │   │   ├── EmpresaConfigDesign/    # Configuraciones generales de la empresa y moneda
    │   │   ├── ImpresorasDesing/       # Ajustes de impresoras de tickets térmicos
    │   │   ├── LandingPages/           # Vistas de bienvenida y onboarding
    │   │   ├── LoginDesing/            # Formularios de acceso y animaciones temáticas
    │   │   ├── POSDesing/              # Núcleo del Punto de Venta (Teclado, Pantalla Cobro, etc.)
    │   │   │   └── CajaDesing/         # Operaciones de caja (Apertura, Conteo Ciego, Arqueo)
    │   │   ├── SucursalesDesign/       # Gestión de sedes y puntos de venta
    │   │   ├── UsuariosDesing/         # Matriz de permisos de usuario
    │   │   ├── formularios/            # 14 formularios modales con validaciones reactivas
    │   │   ├── sidebar/                # Menús de navegación de escritorio y móvil responsive
    │   │   └── tablas/                 # 14 tablas de datos TanStack Table v8 con paginación
    │   ├── templates/                  # Plantillas de distribución espacial de cada página (19)
    │   └── ui/                         # Componentes de micro-interacción animados reutilizables
    ├── context/                        # Proveedores de contexto de React (AuthContext)
    ├── hooks/                          # Hooks personalizados de lógica de negocio y seguridad (12)
    ├── pages/                          # Vistas asociadas a los endpoints del enrutador (22)
    ├── reports/                        # Generadores de comprobantes en PDF térmico con pdfmake (4)
    ├── routers/                        # Definición de rutas declarativas protegidas (routes.jsx)
    ├── store/                          # 32 Stores de Zustand para manejo de estado global
    ├── styles/                         # Estilos globales, paletas Halloween (Light/Dark) y variabl
es
    ├── supabase/                       # 27 Módulos de llamada directa a API y funciones RPC
    ├── tanstack/                       # 24 Módulos de hooks TanStack Query (Queries & Mutations)
    └── utils/                          # Utilidades criptográficas QZ, conversiones de moneda y fue
ntes
```

CAPÍTULO 2

Desglose Detallado Archivo por Archivo

Inventario técnico y enciclopedia de archivos del sistema. Se describe la responsabilidad arquitectónica, símbolos exportados, relaciones de importación y patrones de ejecución internos de cada archivo del proyecto.

2.1 Archivos Raíz, Entorno y Configuración del Ecosistema

Config & Build

`package.json`

CONFIGURACIÓN / DEPENDENCIAS

Manifiesto principal de la aplicación Node/Vite. Define los scripts de ejecución ('dev', 'build', 'lint', 'preview') y el árbol completo de 36 dependencias productivas (React 19, Supabase JS, TanStack Query/Table, Zustand, Styled-Components, QZ-Tray, jsrsasign, pdfmake, Ant Design, Framer Motion) y herramientas de desarrollo.

Exporta:	`Objeto JSON con metadata del proyecto, dependencias y scripts de automatización.`
Dependencias:	`Node.js, npm/yarn.`
Lógica Clave:	Configura 'type': 'module' para habilitar ESM nativo y fija la versión 19 de React junto a Vite 7.

`vite.config.js`

BUILD TOOLING / VITE

Archivo de configuración del empaquetador Vite. Integra el plugin oficial '@vitejs/plugin-react' para compilación ultrarrápida mediante Fast Refresh y define opciones de optimización de chunks para la suite de producción.

Exporta:	`export default defineConfig({ ... })`
Dependencias:	`vite, @vitejs/plugin-react`
Lógica Clave:	Habilita compilación JSX transform y gestión de hot reloading sin recargas completas de navegador.

`eslint.config.js`

LINTER / CALIDAD DE CÓDIGO

Configuración moderna de ESLint en formato Flat Config. Aplica reglas recomendadas de JavaScript, React Hooks ('react-hooks/rules-of-hooks', 'react-hooks/exhaustive-deps') y React Refresh para mantener la integridad reactiva.

Exporta:	`Configuración exportada bajo formato Flat Config ESM.`
Dependencias:	`@eslint/js, eslint-plugin-react-hooks, eslint-plugin-react-refresh, globals`
Lógica Clave:	Previene dependencias cíclicas y variables no utilizadas, asegurando la consistencia del código.

`firebase.json`

HOSTING / DESPLIEGUE

Reglas de despliegue para Firebase Hosting. Configura la reescritura de URLs para soportar Single Page Application (SPA), redirigiendo todas las rutas entrantes ('**') hacia 'index.html' para que React Router resuelva las vistas.

Exporta:	`Reglas JSON de hosting con rewrites hacia /index.html.`
Dependencias:	`Firebase CLI.`
Lógica Clave:	Garantiza que rutas profundas como '/pos' o '/configuracion/productos' no generen error HTTP 404 al refrescar.

`index.html`

ENTRADA WEB / DOM HOST

Documento HTML5 de entrada de la aplicación web. Contiene el viewport responsive, enlaces de metadatos, favicon y el contenedor raíz '#root' sobre el cual React monta la jerarquía de componentes.

Exporta:	`Documento HTML5 estándar.`
Dependencias:	`src/main.jsx`
Lógica Clave:	Carga modularmente '/src/main.jsx' mediante un tag de script con 'type="module"'.

`.env`

VARIABLES DE ENTORNO

Almacena las credenciales públicas de conexión con la infraestructura en la nube de Supabase: 'VITE_SUPABASE_URL' y 'VITE_SUPABASE_PUBLISHABLE_DEFAULT_KEY'.

Exporta:	`Variables de entorno con prefijo VITE_.`
Dependencias:	`import.meta.env en Vite.`
Lógica Clave:	Permite alternar entre entornos locales, staging o producción sin modificar el código fuente.

`supabase/config.toml`

BACKEND / CONFIG CLI SUPABASE

Archivo de configuración para la CLI local de Supabase. Declara el identificador de proyecto ('sistema-ventas-pos'), puertos locales de PostgreSQL, API REST (PostgREST), Storage y Studio.

Exporta:	`Configuración TOML de infraestructura Supabase.`
Dependencias:	`Supabase CLI, Docker.`
Lógica Clave:	Permite levantar y replicar el entorno de desarrollo local con emuladores idénticos a producción.

`supabase/functions/enviar-facturas/index.ts` EDGE FUNCTION / MICROSERVICIO DENO

Función serverless escrita en TypeScript que corre sobre Deno en el borde de Supabase. Consulta las empresas suscritas en la base de datos, ensambla facturas electrónicas profesionales en PDF utilizando 'pdf-lib' (dibujo vectorial de cabeceras, tablas y datos fiscales) y despacha los correos con el PDF adjunto en Base64 utilizando la API de Resend.

Exporta:	`serve(async (req) ⇒ Response)`
Dependencias:	`https://deno.land/std/http/server.ts, @supabase/supabase-js, pdf-lib, Resend API`
Lógica Clave:	Procesamiento asíncrono server-side, diseño de comprobante A4 con tipografías vectoriales Helvetica y despacho por SMTP REST.

2.2 Núcleo de Entrada y Sistema de Enrutamiento Core & Routing

`src/main.jsx` PUNTO DE MONTAJE REACT

Punto de inicialización del DOM de React 19. Crea el root con 'createRoot', envuelve la aplicación en 'StrictMode', 'BrowserRouter' de React Router DOM y 'QueryClientProvider' de TanStack Query para inicializar el motor de caché.

Exporta:	`Punto de ejecución de inicio de la SPA.`
Dependencias:	`react, react-dom/client, react-router-dom, @tanstack/react-query, App.jsx`
Lógica Clave:	Instancia un 'QueryClient' central que gobierna todas las peticiones asíncronas y monta ''.

`src/App.jsx` ORQUESTADOR DE PROVIDERS Y TEMA

Componente de nivel superior. Sincroniza dinámicamente el tema visual (Light / Dark con paletas Halloween) según la preferencia del usuario en 'useUsuariosStore', provee el 'ThemeProvider' de Styled-Components, el 'AuthContextProvider', los estilos globales, el observador de limpieza 'LimpiezaVentas', la compuerta 'AuthGate' y las herramientas 'ReactQueryDevtools'.

Exporta:	`export default App`
Dependencias:	`styled-components, AuthContext, ThemeStore, UsuariosStore, LimpiezaVentas, AuthGate, Myroutes, Devtools`
Lógica Clave:	Efecto 'useEffect' reactivo que conmuta el estilo del tema de interfaz cuando cambia el campo 'tema' del usuario.

`src/index.js` BARREL FILE CENTRAL (>250 EXPORTS)

Archivo de índice y exportación masiva del proyecto. Centraliza y re-exporta todos los componentes atómicos, moleculares, organismos, templates, páginas, hooks, reportes, utilidades y stores de Zustand para permitir importaciones limpias de primer nivel en cualquier parte del código.

Exporta:	`Más de 250 exports agrupados.`
Dependencias:	`Toda la jerarquía de src/`
Lógica Clave:	Elimina las rutas relativas complejas tipo '../../../' facilitando la refactorización y mantenibilidad.

`src/routers/routes.jsx` ENRUTADOR DECLARATIVO Y PERMISOS

Define el árbol completo de rutas URL de la aplicación bajo 'AnimatePresence' para transiciones fluidas de página. Encapsula cada ruta protegida dentro de '' y '', clasificando los accesos en 'authenticated' (POS, Dashboard, Inventario, Clientes, etc.) y 'non-authenticated' (Login).

Exporta:	`export function Myroutes()`
Dependencias:	`react-router-dom, framer-motion, ProtectedRoute, Layout, Pages`
Lógica Clave:	Gobierna la navegación SPA y delega la inspección de permisos de módulo a 'ProtectedRoute' antes de renderizar la vista.

2.3 Capa de Contextos y Hooks Personalizados Context & Hooks

`src/context/AuthContext.jsx` CONTEXTO DE AUTENTICACIÓN SUPABASE

Proveedor de contexto que escucha el estado de autenticación en tiempo real mediante 'supabase.auth.onAuthStateChange'. Gestiona el estado 'user' y la bandera 'isCheckingAuth'. Al detectar una nueva sesión, invoca en un 'setTimeout(..., 0)' la sincronización de la empresa del usuario para evitar bloqueos del hilo principal de Supabase.

Exporta:	`AuthContextProvider, userAuth()`
Dependencias:	`react, supabase.config, crudUsuarios, EmpresaStore`
Lógica Clave:	Mantiene la sesión sincronizada con localStorage y dispara la inicialización de la empresa en la primera entrada.

`src/hooks/AuthGate.jsx` COMPUERTA PROTECTORA DE SESIÓN

Componente interceptor que impide parpadeos de interfaz. Mientras 'isCheckingAuth' o 'isSigningOut' están activos, superpone un overlay animado con 'Spinner1' respetando el color del tema activo, liberando el contenido solo cuando la sesión está verificada con certeza.

Exporta:	`export const AuthGate = ({ children })`
Dependencias:	`react, styled-components, framer-motion, AuthContext, AuthStore, Spinner1`
Lógica Clave:	Posee un timer de salvaguarda de 2500ms para forzar la liberación del overlay en caso de pérdida de red.

`src/hooks/ProtectedRoute.jsx` GUARDIA DE RUTAS Y RBAC DINÁMICO

Controlador de acceso a nivel de ruta. Evalúa si el usuario está autenticado y consulta la lista de permisos globales ('useMostrarPermisosGlobalesQueryStack'). Valida si la ruta actual 'location.pathname' coincide con un módulo autorizado; si no, redirige inmediatamente a '/sin-permiso'.

Exporta:	`export const ProtectedRoute = ({ children, accesby })`
Dependencias:	`react-router-dom, AuthContext, UsuariosStore, PermisosStack, Spinners`
Lógica Clave:	Asegura que un cajero no pueda acceder escribiendo directamente en la URL a '/configuracion/empresa' o módulos vedados.

src/hooks/UseValidarPermisosOpertivos.jsx

VALIDADOR DE PERMISOS OPERATIVOS

Hook que expone la función 'validarPermiso(nombreAccion)'. Valida permisos específicos en caliente (ej: 'Cobrar venta', 'Eliminar ítems') contra la lista de permisos del usuario activo, mostrando advertencias SweetAlert2 si el privilegio es denegado.

Exporta:	`export const useValidarPermisosOpertivos = () ⇒ { validarPermiso }`
Dependencias:	`PermisosStore, sweetalert2`
Lógica Clave:	Permite proteger botones individuales y disparadores de eventos sin necesidad de alterar las rutas.

src/hooks/GenerarCodigo.jsx

GENERADOR DE CÓDIGOS SKU / BARRAS

Hook de utilidad para generar códigos únicos secuenciales o aleatorios para nuevos productos, clientes o series de tickets cuando no se ingresa un código de barras de fábrica.

Exporta:	`export const useGenerarCodigo = () ⇒ { ... }`
Dependencias:	`react`
Lógica Clave:	Genera secuencias alfanuméricas compatibles con lectores de códigos de barras ópticos estándar.

src/hooks/Layout.jsx

LAYOUT MAESTRO DE LA APLICACIÓN

Estructura visual envolvente que organiza la interfaz administrativa. Renderiza el 'Sidebar' persistente en pantallas de escritorio, el 'MenuMovil' en dispositivos táctiles, el 'IndicadorConexion' en tiempo real y el contenedor dinámico del contenido hijo.

Exporta:	`export const Layout = ({ children })`
Dependencias:	`react, styled-components, Sidebar, MenuMovil, IndicadorConexion`
Lógica Clave:	Controla la distribución responsive en columnas y los márgenes de contenido según el dispositivo.

src/hooks/LimpiezaVentas.jsx

WATCHER DE LIMPIEZA DE VENTAS HUÉRFANAS

Efecto de limpieza en background. Se ejecuta al montar la aplicación para detectar y eliminar ventas en estado 'pendiente' abandonadas por desconexión o recarga accidental en la caja actual.

Exporta:	`export const LimpiezaVentas = () ⇒ null`
Dependencias:	`react, VentasStack, CierreCajaStore`
Lógica Clave:	Garantiza que no queden ítems reservados en base de datos de ventas que nunca llegaron a confirmarse.

src/hooks/useCerrarSesion.jsx

CONTROLADOR DE LOGOUT TRANSICIONAL

Orquesta el proceso ordenado de desconexión: activa la bandera 'isSigningOut' en 'AuthStore', muestra el overlay temático, ejecuta 'supabase.auth.signOut()', limpia los stores de Zustand y redirige a '/login'.

Exporta:	`export const useCerrarSesion = () ⇒ { cerrarSesion }`
Dependencias:	`AuthStore, supabase.config, react-router-dom`
Lógica Clave:	Previene inconsistencias de estado y fugas de memoria al cerrar la sesión de un cajero.

src/hooks/useEstadoConexion.js

MONITOR DE CONECTIVIDAD A INTERNET

Escucha los eventos globales del navegador 'online' y 'offline'. Provee el estado booleano de conexión a la nube y alerta visualmente si el cajero pierde acceso a la red de Supabase.

Exporta:	`export const useEstadoConexion = () ⇒ { isOnline }`
Dependencias:	`react, sonner`
Lógica Clave:	Emite notificaciones toast automáticas advirtiendo que los cambios no podrán sincronizarse sin internet.

src/hooks/useFormattedDate.jsx

FORMATEADOR DE FECHA LOCAL

Hook que retorna la fecha y hora actual formateada en el estándar ISO requerido por las funciones RPC de Supabase y los comprobantes fiscales ('YYYY-MM-DD HH:mm:ss').

Exporta:	`export const useFormattedDate = () ⇒ string`
Dependencias:	`dayjs / date-fns`
Lógica Clave:	Estandariza las marcas temporales de ventas, aperturas y movimientos de caja.

src/hooks/useImageSelector.jsx

SELECTOR Y OPTIMIZADOR DE IMÁGENES

Gestiona la subida de logos corporativos y fotos de productos. Valida tamaño máximo, tipo MIME, genera previsualizaciones locales 'URL.createObjectURL' y convierte las imágenes a base64 o Storage Blobs.

Exporta:	`export const useImageSelector = () ⇒ { ... }`
Dependencias:	`react, sonner`
Lógica Clave:	Evita el envío de archivos corruptos o excesivamente pesados a los buckets de almacenamiento de Supabase.

src/hooks/useInputDinero.jsx

MÁSCARA DE ENTRADA DE DIVISAS

Controla los campos de entrada numérica de dinero en el teclado del POS. Aplica separadores de miles, restringe caracteres alfanuméricos y calcula valores limpios en tiempo real para el cálculo del cambio/vuelto.

Exporta:	`export const useInputDinero = () ⇒ { ... }`
Dependencias:	`react, Conversiones`
Lógica Clave:	Garantiza precisión decimal y previene errores tipográficos durante el cobro rápido en caja.

src/hooks/useSupabaseSubscription.jsx

SUSCRIPCIÓN REALTIME A TABLAS

Hook que abre canales WebSocket sobre Supabase Realtime para tablas críticas (como 'ventas' y 'movimientos_caja'). Dispara invalidaciones de queries en TanStack Query automáticamente ante inserciones externas.

Exporta:	`export const useSupabaseSubscription = (tabla, callback) ⇒ void`
Dependencias:	`react, supabase.config, @tanstack/react-query`
Lógica Clave:	Permite que un dashboard gerencial o una segunda pantalla de caja vea reflejadas las ventas al instante.

2.4 Capa de Estado Global del Cliente con Zustand (/store – 32 Stores)

State Management

Zustand gobierna el estado cliente síncrono, persistente y transitorio de la aplicación. Elimina el 'prop drilling' y ofrece un rendimiento superior a Redux o Context al no generar re-renderizados innecesarios.

`src/store/AuthStore.jsx`

CONTROL DE ESTADOS DE TRANSICIÓN DE SESIÓN

Store de Zustand especializado en control de estados de transición de sesión.

Exporta:	`useAuthStore`
Dependencias:	`zustand`
Lógica Clave:	Gestiona las banderas booleanas 'isSigningOut' y los métodos 'iniciarCierreSesion' y 'finalizarCierreSesion' para coordinar las animaciones de overlay en 'AuthGate'.

`src/store/CartVentasStoreTemporal.jsx`

CARRITO DE COMPRAS POS PERSISTENTE

Store de Zustand especializado en carrito de compras pos persistente.

Exporta:	`useCartVentasStoreTemporal`
Dependencias:	`zustand, persist, ClientesProveedoresStore, sonner`
Lógica Clave:	Mantiene los productos agregados a la venta ('items'), calcula el subtotal y total dinámicamente con 'calcularTotal(items)', controla cantidades ('addItem', 'removeItem', 'addCantidadItem', 'restarCantidadItem') y persiste en localStorage.

`src/store/VentasStore.jsx`

GESTOR DE VENTA ACTIVA Y COBRO

Store de Zustand especializado en gestor de venta activa y cobro.

Exporta:	`useVentasStore`
Dependencias:	`zustand, crudVenta, sonner`
Lógica Clave:	Administra el flujo de pago: 'idventa', 'total', 'vuelto', 'restante', 'valoresPago' desglosados por método de pago, apertura de modales 'setStatePantallaCobro' y disparador de la función 'confirmarVenta'.

`src/store/CajaStore.jsx`

GESTOR DE CAJAS FÍSICAS REGISTRADAS

Store de Zustand especializado en gestor de cajas físicas registradas.

Exporta:	`useCajaStore`
Dependencias:	`zustand, crudCaja`
Lógica Clave:	Almacena la lista de cajas de la sucursal ('dataCaja'), caja seleccionada ('cajaSeleIctItem'), estado del modal de creación ('stateCaja') y acción en curso ('editar'/'nuevo').

src/store/CierreCajaStore.jsx

GESTOR DE TURNOS Y ESTADOS DE ARQUEO

Store de Zustand especializado en gestor de turnos y estados de arqueo.

Exporta:	`useCierreCajaStore`
Dependencias:	`zustand, crudCierresCaja`
Lógica Clave:	Controla el ciclo del turno: 'dataCierreCaja' (apertura activa), modales de conteo ciego ('stateConteoCaja'), entradas/salidas de efectivo ('stateIngresoSalida') y cierre formal de caja ('stateCierreCaja').

src/store/ProductosStore.jsx

ESTADO DE CATÁLOGO Y BUSCADOR DE PRODUCTOS

Store de Zustand especializado en estado de catálogo y buscador de productos.

Exporta:	`useProductosStore`
Dependencias:	`zustand, crudProductos`
Lógica Clave:	Gobierna la consulta de productos, el término de búsqueda en vivo ('buscador'), el producto seleccionado para edición ('ProductosItemSelect') y los resultados de autocompletado en el POS.

src/store/StockStore.jsx

GESTOR DE STOCK MULTIALMACÉN Y MODALES DE EXISTENCIAS

Store de Zustand especializado en gestor de stock multialmacén y modales de existencias.

Exporta:	`useStockStore`
Dependencias:	`zustand, crudStock`
Lógica Clave:	Almacena las existencias de un producto distribuidas entre diferentes almacenes ('dataStockXAlmacenesYProducto'), controla el modal selector de almacén alterno y consulta alertas de stock crítico.

src/store/CategoriasStore.jsx

GESTOR DE CATEGORÍAS DE PRODUCTOS

Store de Zustand especializado en gestor de categorías de productos.

Exporta:	`useCategoriasStore`
Dependencias:	`zustand, crudCategorias`
Lógica Clave:	Maneja el CRUD de familias/categorías: lista ('datacategorias'), categoría activa, buscador y estado del formulario modal de registro de categoría.

src/store/ClientesProveedoresStore.jsx

DIRECTORIO DE CLIENTES Y PROVEEDORES

Store de Zustand especializado en directorio de clientes y proveedores.

Exporta:	`useClientesProveedoresStore`
Dependencias:	`zustand, crudClientesProveedores`
Lógica Clave:	Diferencia entidades por 'tipo' ('cliente' o 'proveedor'), maneja la selección del cliente actual en el POS ('cliproItemSelect') y búsquedas por documento o nombre.

src/store/CreditosStore.jsx

CONTROL DE CUENTAS POR COBRAR Y CRÉDITOS

Store de Zustand especializado en control de cuentas por cobrar y créditos.

Exporta:	`useCreditosStore`
Dependencias:	`zustand, crudCreditos`
Lógica Clave:	Registra las ventas a crédito pendientes de pago ('datacreditos'), crédito seleccionado para abono ('creditosItemSelect') y filtros de búsqueda de deudores.

src/store/MovimientosCreditosStore.jsx

HISTORIAL DE ABONOS Y TICKETS DE CRÉDITO

Store de Zustand especializado en historial de abonos y tickets de crédito.

Exporta:	`useMovimientosCreditosStore`
Dependencias:	`zustand, crudMovimientosCreditos`
Lógica Clave:	Gobierna el modal de historial de abonos ('isHistorialOpen'), visualización de tickets de crédito ('isTicketCreditoOpen') y formulario de recepción de abonos ('stateIngresoCredito').

src/store/AlmacenesStore.jsx

GESTOR DE BODEGAS Y ALMACENES FÍSICOS

Store de Zustand especializado en gestor de bodegas y almacenes físicos.

Exporta:	`useAlmacenesStore`
Dependencias:	`zustand, crudAlmacenes`
Lógica Clave:	Maneja la lista de almacenes por empresa ('dataAlmacenesXEmpresa') y por sucursal ('dataalmacenxsucursal'), modal de creación y selección para despacho de inventario.

src/store/SucursalesStore.jsx

GESTOR DE SUCURSALES Y SEDES COMERCIALES

Store de Zustand especializado en gestor de sucursales y sedes comerciales.

Exporta:	`useSucursalesStore`
Dependencias:	`zustand, crudSucursales`
Lógica Clave:	Administra las diferentes sedes físicas ('dataSucursales'), sucursal activa seleccionada ('sucursalesItemSelect') y modal de configuración de sucursales.

src/store/AsignacionCajaSucursales.jsx

ASIGNACIÓN DE CAJAS Y SUCURSALES A USUARIOS

Store de Zustand especializado en asignación de cajas y sucursales a usuarios.

Exporta:	`useAsignacionCajaSucursalesStore`
Dependencias:	`zustand, crudAsignacionSucursales`
Lógica Clave:	Controla qué usuarios tienen permiso para operar determinadas cajas en sucursales asignadas ('datSucursalesAsignadas' y 'dataUsuariosAsignados').

src/store/DetalleVentasStore.jsx

GESTOR DEL DETALLE DE VENTA Y ESTADÍSTICAS DE CARRITO

Store de Zustand especializado en gestor del detalle de venta y estadísticas de carrito.

Exporta:	`useDetalleVentasStore`
Dependencias:	`zustand, crudDetalleVenta`
Lógica Clave:	Almacena las líneas de la venta en curso ('detalleventa'), cantidad de ventas, porcentajes de cambio, cálculo del total y reseteo reactivo.

src/store/EmpresaStore.jsx

PARÁMETROS Y CONFIGURACIÓN CORPORATIVA

Store de Zustand especializado en parámetros y configuración corporativa.

Exporta:	`useEmpresaStore`
Dependencias:	`zustand, crudEmpresa`
Lógica Clave:	Guarda los datos de la empresa activa ('dataempresa'): nombre legal, NIT/RUT, logo corporativo, dirección, régimen fiscal y métodos de actualización.

src/store/DashboardStore.jsx

FILTROS Y RANGO DE FECHAS DEL DASHBOARD

Store de Zustand especializado en filtros y rango de fechas del dashboard.

Exporta:	`useDashboardStore`
Dependencias:	`zustand`
Lógica Clave:	Centraliza el rango de fechas ('fechaInicio', 'fechaFin') aplicado a los gráficos analíticos y reportes ejecutivos de ventas y ganancias.

src/store/ImpresorasStore.jsx

CONFIGURACIÓN DE IMPRESORAS TÉRMICAS POR CAJA

Store de Zustand especializado en configuración de impresoras térmicas por caja.

Exporta:	`useImpresorasStore`
Dependencias:	`zustand, crudImpresoras`
Lógica Clave:	Guarda la configuración de impresión asociada a la caja activa ('dataImpresorasXCaja') y la bandera 'statePrintDirecto' para bypass de diálogo de impresión.

src/store/MetodosPagoStore.jsx

CATÁLOGO DE FORMAS DE PAGO

Store de Zustand especializado en catálogo de formas de pago.

Exporta:	`useMetodosPagoStore`
Dependencias:	`zustand, crudMetodoPago`
Lógica Clave:	Almacena los métodos de cobro habilitados (Efectivo, Tarjeta, Transferencia, Crédito, Mixto), iconos representativos y métodos para agregar nuevas pasarelas.

src/store/ModulosStore.jsx

CATÁLOGO DE MÓDULOS DEL SISTEMA

Store de Zustand especializado en catálogo de módulos del sistema.

Exporta:	`useModulosStore`
Dependencias:	`zustand, crudModulos`
Lógica Clave:	Mantiene la lista maestra de todos los módulos disponibles en el sistema ('dataModulos') para su asignación en la matriz de permisos de usuarios.

src/store/MonedaStore.jsx

CONFIGURACIÓN DE MONEDA ACTIVA

Store de Zustand especializado en configuración de moneda activa.

Exporta:	`useMonedaStore`
Dependencias:	`zustand, crudMoneda`
Lógica Clave:	Almacena el símbolo, código ISO (ej: 'COP', 'USD') y formato decimal activo en la empresa para formatear las cifras financieras de toda la plataforma.

src/store/MonedasStore.jsx

BUSCADOR GLOBAL DE PAÍSES Y DIVISAS

Store de Zustand especializado en buscador global de países y divisas.

Exporta:	`useMonedasStore`
Dependencias:	`zustand`
Lógica Clave:	Permite explorar el catálogo internacional de monedas e ISOs para asignar la configuración regional durante el onboarding de la empresa.

src/store/MovCajaStore.jsx

CONTROL DE FLUJO DE EFECTIVO EN CAJA

Store de Zustand especializado en control de flujo de efectivo en caja.

Exporta:	`useMovCajaStore`
Dependencias:	`zustand, crudMovimientosCaja`
Lógica Clave:	Gestiona el registro de movimientos manuales de caja ('insertarMovcaja'), filtrado por fechas y cálculo de totales en efectivo vs métodos electrónicos.

src/store/MovStockStore.jsx

KARDEX DE MOVIMIENTOS DE INVENTARIO

Store de Zustand especializado en kardex de movimientos de inventario.

Exporta:	`useMovStockStore`
Dependencias:	`zustand, crudMovimientosStock`
Lógica Clave:	Controla la trazabilidad del stock: entradas, salidas, ajustes por merma y transferencias entre bodegas con razones de movimiento.

src/store/PermisosStore.jsx

MATRIZ DE PRIVILEGIOS Y SEGURIDAD RBAC

Store de Zustand especializado en matriz de privilegios y seguridad rbac.

Exporta:	`usePermisosStore`
Dependencias:	`zustand, crudPermisos`
Lógica Clave:	Mantiene la lista de permisos globales asignados al usuario en sesión ('dataPermisosGlobales') y los permisos específicos del módulo de configuraciones.

src/store/ResumenVentaStore.jsx

MODAL DE ÉXITO Y RESUMEN POST-COBRO

Store de Zustand especializado en modal de éxito y resumen post-cobro.

Exporta:	`useResumenVentaStore`
Dependencias:	`zustand`
Lógica Clave:	Controla la ventana emergente tras finalizar el cobro: muestra el total pagado, el cambio entregado y dispara el botón de reimpresión de comprobante.

src/store/RolesStore.jsx

CATÁLOGO DE ROLES DE USUARIO

Store de Zustand especializado en catálogo de roles de usuario.

Exporta:	`useRolesStore`
Dependencias:	`zustand, crudRol`
Lógica Clave:	Gestiona los perfiles de usuario (Administrador, Cajero, Supervisor) y el rol asignado en la pantalla de edición de usuarios.

src/store/SerealizacionesStore.jsx

CONSECUTIVOS Y SERIES DE FACTURACIÓN

Store de Zustand especializado en consecutivos y series de facturación.

Exporta:	`useSerealizacionesStore`
Dependencias:	`zustand, crudSerealizaciones`
Lógica Clave:	Controla los tipos de comprobantes autorizados (Ticket, Boleta, Factura), series fiscales, correlativos actuales y el comprobante por defecto en ventas.

src/store/ThemeStore.jsx

GESTOR DE MODO VISUAL HALLOWEEN / LIGHT / DARK

Store de Zustand especializado en gestor de modo visual halloween / light / dark.

Exporta:	`useThemeStore`
Dependencias:	`zustand`
Lógica Clave:	Almacena el tema activo ('light' o 'dark') y los estilos CSS del tema de Styled-Components, permitiendo cambiar el skin visual en toda la aplicación.

`src/store/UsuariosStore.jsx` GESTOR DE PERSONAL Y CAJEROS

Store de Zustand especializado en gestor de personal y cajeros.

Exporta:	`useUsuariosStore`
Dependencias:	`zustand, crudUsuarios`
Lógica Clave:	Mantiene los datos del perfil del usuario en sesión ('datausuarios'), avatar, asignación de empresa y operaciones CRUD de gestión de usuarios.

`src/store/GlobalStore.jsx` VARIABLES GLOBALES Y MODALES COMPARTIDOS

Store de Zustand especializado en variables globales y modales compartidos.

Exporta:	`useGlobalStore`
Dependencias:	`zustand`
Lógica Clave:	Maneja banderas genéricas compartidas entre componentes: 'stateClose', 'isExploding' (animación de confeti), archivo en buffer temporal y URL de imagen.

`src/store/ContraseaStore.jsx` GESTOR TEMPORAL DE CLAVES DE ACCESO

Store de Zustand especializado en gestor temporal de claves de acceso.

Exporta:	`useContraseaStore`
Dependencias:	`zustand, crudContrasea`
Lógica Clave:	Módulo auxiliar de verificación de credenciales de seguridad para acciones administrativas sensibles.

2.5 Capa de Integración y Procedimientos Almacenados Supabase (/supabase – 27 Archivos)

Backend API & RPC

Módulos de bajo nivel que interactúan directamente con el cliente de Supabase (PostgreSQL). Encapsulan consultas a tablas bajo RLS y la ejecución de procedimientos almacenados (RPC) para transacciones atómicas.

`src/supabase/supabase.config.jsx` CLIENTE CENTRAL SUPABASE

Módulo de acceso a datos para cliente central supabase.

Exporta:	`supabase`
Dependencias:	`@supabase/supabase-js, import.meta.env`
Lógica Clave:	Inicializa la instancia 'createClient' con la URL y la llave anónima/publicable leídas desde las variables de entorno.

src/supabase/crudVenta.jsx

TRANSACCIONES DE VENTA Y RPC

Módulo de acceso a datos para transacciones de venta y rpc.

Exporta:	`InsertarVentas, EliminarVentasIncompletas, ConfirmarVenta, EliminarVenta, MostrarVentaCompletaPorId`
Dependencias:	`supabase.config`
Lógica Clave:	Invoca el RPC 'confirmar_venta' para ejecutar la transacción atómica de venta y 'mostrar_venta_completa_por_id' para tickets.

src/supabase/crudDetalleVenta.jsx

LÍNEAS DE DETALLE DE VENTA

Módulo de acceso a datos para líneas de detalle de venta.

Exporta:	`InsertarDetalleVenta, MostrarDetalleVenta, EliminarDetalleVenta, EditarCantidadDetalleVenta`
Dependencias:	`supabase.config`
Lógica Clave:	Gestiona los ítems específicos incluidos en una venta, calculando subtotales y vinculándolos con la cabecera de la venta.

src/supabase/crudProductos.jsx

CATÁLOGO DE PRODUCTOS Y BÚSQUEDAS

Módulo de acceso a datos para catálogo de productos y búsquedas.

Exporta:	`InsertarProductos, MostrarProductos, BuscarProductos, EliminarProductos, EditarProductos`
Dependencias:	`supabase.config`
Lógica Clave:	Ejecuta los RPCs 'insertarproductos', 'mostrarproductos', 'buscarproductos' y 'editarproductos' para consultas rápidas indexadas.

src/supabase/crudStock.jsx

INVENTARIO FÍSICO Y ALERTAS

Módulo de acceso a datos para inventario físico y alertas.

Exporta:	`MostrarStock, MostrarStockAlmacenesYProducto, MostrarAlertasStockXVenta`
Dependencias:	`supabase.config`
Lógica Clave:	Consulta existencias actuales, existencias distribuidas por bodegas y dispara alertas cuando el stock es inferior al mínimo.

src/supabase/crudCaja.jsx

TERMINALES DE CAJA

Módulo de acceso a datos para terminales de caja.

Exporta:	`InsertarCajas, MostrarCajas, EliminarCajas, EditarCajas`
Dependencias:	`supabase.config`
Lógica Clave:	Gestiona las cajas registradoras físicas creadas dentro de una sucursal.

src/supabase/crudCierresCaja.jsx

TURNOS, APERTURAS Y CIERRES

Módulo de acceso a datos para turnos, aperturas y cierres.

Exporta:	`AperturarCaja, MostrarAperturaCajaPorUsuario, CerrarCaja`
Dependencias:	`supabase.config`
Lógica Clave:	Invoca el RPC 'aperturarcaja' para iniciar turno y 'cerrarcaja' para registrar el arqueo ciego final y diferencias.

src/supabase/crudMovimientosCaja.jsx

FLUJO DE EFECTIVO EN CAJA

Módulo de acceso a datos para flujo de efectivo en caja.

Exporta:	`InsertarMovimientosCaja, MostrarMovimientosCajaXEmpresaYFecha, MostrarVentasMetodoPago`
Dependencias:	`supabase.config`
Lógica Clave:	Registra transacciones de entrada/salida de efectivo y pagos de ventas desglosados por método de pago.

src/supabase/crudAlmacenes.jsx

BODEGAS Y ALMACENES

Módulo de acceso a datos para bodegas y almacenes.

Exporta:	`InsertarAlmacenes, MostrarAlmacenesXSucursal, MostrarAlmacenesXEmpresa, EliminarAlmacenes`
Dependencias:	`supabase.config`
Lógica Clave:	Maneja la creación y listado de almacenes vinculados a sucursales de la empresa.

src/supabase/crudSucursales.jsx

SEDES Y PUNTOS DE VENTA

Módulo de acceso a datos para sedes y puntos de venta.

Exporta:	`InsertarSucursales, MostrarSucursales, EliminarSucursales, EditarSucursales`
Dependencias:	`supabase.config`
Lógica Clave:	Gobierna la estructura multi-sucursal de la empresa.

src/supabase/crudAsignacionSucursales.jsx

ASIGNACIÓN DE PERSONAL A CAJAS

Módulo de acceso a datos para asignación de personal a cajas.

Exporta:	`MostrarSucursalesAsignadas, AsignarUsuarioSucursal`
Dependencias:	`supabase.config`
Lógica Clave:	Vincula usuarios y cajeros con sucursales específicas autorizadas.

src/supabase/crudCategorias.jsx

FAMILIAS DE PRODUCTOS

Módulo de acceso a datos para familias de productos.

Exporta:	`InsertarCategorias, MostrarCategorias, BuscarCategorias, EliminarCategorias, EditarCategorias`
Dependencias:	`supabase.config`
Lógica Clave:	Gestiona el árbol de categorías con nombres, iconos y colores de identificación.

src/supabase/crudClientesProveedores.jsx

TERCEROS: CLIENTES Y PROVEEDORES

Módulo de acceso a datos para terceros: clientes y proveedores.

Exporta:	`InsertarClientesProveedores, MostrarClientesProveedores, BuscarClientes, EliminarClientesProveedores`
Dependencias:	`supabase.config`
Lógica Clave:	Mantiene el padrón de clientes (para emisión de tickets y créditos) y proveedores (para compras e inventario).

src/supabase/crudCreditos.jsx

CUENTAS POR COBRAR DE CLIENTES

Módulo de acceso a datos para cuentas por cobrar de clientes.

Exporta:	`MostrarCreditos, BuscarCreditos, InsertarCredito`
Dependencias:	`supabase.config`
Lógica Clave:	Controla las deudas de clientes originadas por ventas a crédito con saldo pendiente y fecha límite.

src/supabase/crudMovimientosCreditos.jsx

ABONOS Y RECIBOS DE CAJA DE CRÉDITOS

Módulo de acceso a datos para abonos y recibos de caja de créditos.

Exporta:	`InsertarAbonoCredito, MostrarMovimientosCredito, MostrarVentaCompletaCredito`
Dependencias:	`supabase.config`
Lógica Clave:	Invoca el RPC 'insertarabonocredito' que disminuye el saldo pendiente y emite el recibo de caja.

src/supabase/crudEmpresa.jsx

DATOS CORPORATIVOS Y RÉGIMEN FISCAL

Módulo de acceso a datos para datos corporativos y régimen fiscal.

Exporta:	`InsertarEmpresa, MostrarEmpresa, EditarEmpresa, EditarMonedaEmpresa`
Dependencias:	`supabase.config`
Lógica Clave:	Actualiza el perfil de la empresa, logotipo, moneda base y configuración fiscal.

src/supabase/crudImpresoras.jsx

ASIGNACIÓN DE IMPRESORAS

Módulo de acceso a datos para asignación de impresoras.

Exporta:	`MostrarImpresorasXCaja, GuardarImpresoraCaja`
Dependencias:	`supabase.config`
Lógica Clave:	Asocia nombres de impresoras térmicas de red o USB a terminales de caja específicas.

src/supabase/crudMetodoPago.jsx

FORMAS DE PAGO HABILITADAS

Módulo de acceso a datos para formas de pago habilitadas.

Exporta:	`InsertarMetodosPago, MostrarMetodosPago, EditarMetodosPago, EliminarMetodosPago`
Dependencias:	`supabase.config`
Lógica Clave:	Administra los medios de pago aceptados y sus iconos asociados en el teclado del POS.

src/supabase/crudModulos.jsx

MÓDULOS DE SISTEMA

Módulo de acceso a datos para módulos de sistema.

Exporta:	`MostrarModulos`
Dependencias:	`supabase.config`
Lógica Clave:	Consulta los módulos instalados para el panel de asignación de permisos de usuarios.

src/supabase/crudMoneda.jsx

MONEDA Y SÍMBOLO FISCAL

Módulo de acceso a datos para moneda y símbolo fiscal.

Exporta:	`MostrarMoneda`
Dependencias:	`supabase.config`
Lógica Clave:	Obtiene el formato monetario asignado a la empresa.

src/supabase/crudMovimientosStock.jsx

KARDEX DE INVENTARIO

Módulo de acceso a datos para kardex de inventario.

Exporta:	`InsertarMovimientoStock, MostrarMovimientosStock`
Dependencias:	`supabase.config`
Lógica Clave:	Inserta auditorías de stock ante ajustes, mermas o inventario físico inicial.

src/supabase/crudPermisos.jsx

PRIVILEGIOS RBAC POR USUARIO

Módulo de acceso a datos para privilegios rbac por usuario.

Exporta:	`MostrarPermisosPorUsuario, MostrarPermisosGlobales, GuardarPermisos`
Dependencias:	`supabase.config`
Lógica Clave:	Consulta y actualiza la relación usuario-módulos para gobernar las rutas y botones protegidos.

src/supabase/crudRol.jsx

ROLES DE USUARIO

Módulo de acceso a datos para roles de usuario.

Exporta:	`MostrarRoles`
Dependencias:	`supabase.config`
Lógica Clave:	Obtiene los perfiles preconfigurados en la base de datos.

src/supabase/crudSerealizaciones.jsx

SERIES Y CORRELATIVOS FISCALES

Módulo de acceso a datos para series y correlativos fiscales.

Exporta:	`MostrarSerealizaciones, EditarSerealizacionDefault, InsertarSerealizacion`
Dependencias:	`supabase.config`
Lógica Clave:	Gestiona números correlativos automáticos de facturación ('F001-0001', 'T001-0001').

src/supabase/crudTipoDocumentos.jsx

DOCUMENTOS DE IDENTIDAD

Módulo de acceso a datos para documentos de identidad.

Exporta:	`MostrarTipoDocumentos`
Dependencias:	`supabase.config`
Lógica Clave:	Catálogo de tipos de documento de identidad (CC, NIT, DNI, Pasaporte).

src/supabase/crudUsuarios.jsx

GESTIÓN DE PERSONAL

Módulo de acceso a datos para gestión de personal.

Exporta:	`MostrarUsuarios, InsertarUsuarios, EditarUsuario, EditarTema`
Dependencias:	`supabase.config`
Lógica Clave:	Administra cajeros y administradores, actualizando además el tema visual preferido (Light/Dark).

`src/supabase/crudContrasea.jsx` CLAVES ADMINISTRATIVAS

Módulo de acceso a datos para claves administrativas.

Exporta:	`VerificarClave`
Dependencias:	`supabase.config`
Lógica Clave:	Valida contraseñas maestras para autorizaciones directas en caja.

2.6 Capa de Sincronización y Mutaciones TanStack Query (/tanstack - 24 Módulos)

React Query v5

Hooks de alto nivel que implementan consultas reactivas con caché inteligente y mutaciones con invalidación automática de consultas de red.

`src/tanstack/VentasStack.jsx` ORQUESTADOR DE MUTACIONES Y CONSULTAS DE VENTA

Hook de TanStack Query para orquestador de mutaciones y consultas de venta.

Exporta:	`useConfirmarVentasMutationStack, useEliminarVentasIncompletasMutateStack, useMostrarCantidadVentasQueryStack, useSumarVentasQueryStack, useMostrarGanaciasXEmpresaQueryStack, useMostrarVentasAgrupadasXFechaQueryStack, useMostrarTop5MasVendidosXCantidadQueryStack, useMostrarTop10MasVendidosXMontoQueryStack`
Dependencias:	`@tanstack/react-query, VentasStore, UsuariosStore, CierreCajaStore, SerealizacionesStore, MetodosPagoStore, MovCajaStore, ImpresorasStore, StockStore, ResumenVentaStore, sweetalert2, sonner`
Lógica Clave:	El hook 'useConfirmarVentasMutationStack' valida 'restante === 0', ejecuta el RPC de venta, desglosa los métodos de pago en 'movimientos_caja', envía el comando de apertura de gaveta, ejecuta la impresión térmica, alerta si hay productos con stock crítico y limpia el estado.

`src/tanstack/ProductosStack.jsx` GESTIÓN DE CATÁLOGO Y BUSCADOR EN POS

Hook de TanStack Query para gestión de catálogo y buscador en pos.

Exporta:	`useMostrarProductosQueryStack, useBuscarProductosQueryStack, useBuscarProductosCodigoQueryStack, useInsertarProductosMutationStack`
Dependencias:	`@tanstack/react-query, ProductosStore, crudProductos`
Lógica Clave:	Implementa búsqueda con debounce y autocompletado en el POS mediante código de barras.

`src/tanstack/CajasStack.jsx` MUTACIONES DE CAJAS Y APERTURA DE TURNOS

Hook de TanStack Query para mutaciones de cajas y apertura de turnos.

Exporta:	`useInsertarCajasMutationStack, useElimarCajasMutationStack, useAperturarCajasMutationStack`
Dependencias:	`@tanstack/react-query, CajaStore, CierreCajaStore, crudCaja`
Lógica Clave:	Abre formalmente el turno del cajero e invalida las consultas de cierre de caja.

src/tanstack/CierreCajaStack.jsx

CONSULTAS DE ESTADO DE TURNO

Hook de TanStack Query para consultas de estado de turno.

Exporta:	`useMostrarAperturaCajaPorUsuarioQueryStack, useMostrarCierreCajaPorEmpresaQueryStack`
Dependencias:	`@tanstack/react-query, CierreCajaStore, crudCierresCaja`
Lógica Clave:	Verifica si el usuario actual posee una caja abierta para decidir si renderiza el POS o la pantalla de apertura.

src/tanstack/StockStack.jsx

CONSULTAS DE INVENTARIO MULTIALMACÉN

Hook de TanStack Query para consultas de inventario multialmacén.

Exporta:	`useMostrarStockQueryStack, useMostrarStckAlmacenYProductoQueryStack, useMostrarStockAlmacenesyProductoQueryStack`
Dependencias:	`@tanstack/react-query, StockStore, crudStock`
Lógica Clave:	Consulta existencias en tiempo real por producto en el almacén predeterminado y almacenes alternos.

src/tanstack/AlmacenesStack.jsx

BODEGAS Y DESPACHOS

Hook de TanStack Query para bodegas y despachos.

Exporta:	`useMostrarAlmacenesXSucursalQueryStack, useMostrarAlmacenesXEmpresaQueryStack, useInsertarAlmacenesXSucursalMuattionStack, useEliminarAlmacenesMutationStack`
Dependencias:	`@tanstack/react-query, AlmacenesStore, crudAlmacenes`
Lógica Clave:	Gestiona bodegas y actualiza la lista de selección en el header del POS.

src/tanstack/SucursalesStack.jsx

SEDES DE LA EMPRESA

Hook de TanStack Query para sedes de la empresa.

Exporta:	`useMostrarSucursalesXEmpresaStack, useMostrarCajasPorSucursalQueryStack, useInsertarSucursalesMutationStack, useEliminarSucursalesMutationStack`
Dependencias:	`@tanstack/react-query, SucursalesStore, crudSucursales`
Lógica Clave:	Provee la jerarquía de sucursales e invalida cajas al cambiar de sede.

src/tanstack/AsignacionesSucursal.jsx

PERMISOS DE SUCURSAL

Hook de TanStack Query para permisos de sucursal.

Exporta:	`useMostrarSucursalesAsignadsQueryStack`
Dependencias:	`@tanstack/react-query, AsignacionCajaSucursales, crudAsignacionSucursales`
Lógica Clave:	Valida las sedes asignadas al cajero conectado.

src/tanstack/CategoriasStack.jsx

CATEGORÍAS DE PRODUCTOS

Hook de TanStack Query para categorías de productos.

Exporta:	`useMostrarCategoriasQueryStack, useBuscarCategoriasQueryStack, useInsertarCategoriasMutationStack`
Dependencias:	`@tanstack/react-query, CategoriasStore, crudCategorias`
Lógica Clave:	Sincroniza familias de productos e invalida el catálogo tras inserciones.

src/tanstack/ClientesProveedoresStack.jsx

TERCEROS EN POS Y COMPRAS

Hook de TanStack Query para terceros en pos y compras.

Exporta:	`useMostrarClientesProveedoresQueryStack, useBuscarClientesQueryStack, useInsertarClientesProveedoresMutationStack`
Dependencias:	`@tanstack/react-query, ClientesProveedoresStore, crudClientesProveedores`
Lógica Clave:	Facilita la selección rápida de clientes para facturas o venta a crédito.

src/tanstack/CreditosStack.jsx

FLUJO DE CRÉDITOS Y ABONOS

Hook de TanStack Query para flujo de créditos y abonos.

Exporta:	`useMostrarCreditosQueryStack, useBuscarCreditsoQueryStack, useInsertarAbonoCreditoMuatationStack, useObtenerMovimientosCreditoQueryStack, useMostrarVentaCompletaCreditoQueryStack`
Dependencias:	`@tanstack/react-query, CreditosStore, MovimientosCreditosStore, crudCreditos`
Lógica Clave:	Procesa los abonos de clientes, actualiza saldos pendientes y genera tickets de recibo de caja.

src/tanstack/DetallesVentaStack.jsx

LÍNEAS DE LA VENTA EN CURSO

Hook de TanStack Query para líneas de la venta en curso.

Exporta:	`useMostrarDetalleVentaQueryStack, useEditarCantidadDetalleVentaMutationStack, useEliminarCantidadDetalleVentaMutationStack`
Dependencias:	`@tanstack/react-query, DetalleVentasStore, crudDetalleVenta`
Lógica Clave:	Ajusta cantidades en la pantalla de cobro e invalida las existencias de stock.

src/tanstack/EmpresaStack.jsx

CONFIGURACIÓN DE LA EMPRESA

Hook de TanStack Query para configuración de la empresa.

Exporta:	`useMostrarEmpresaQueryStack, useUpdatEmpresaMutateStack, useUpdatEmpresaTicketMutateStack, useEditarMonedaConfigEmpresaMutationStack`
Dependencias:	`@tanstack/react-query, EmpresaStore, crudEmpresa`
Lógica Clave:	Actualiza el encabezado del ticket, logotipo e información fiscal.

src/tanstack/ImpresorasStack.jsx

IMPRESORAS TÉRMICAS

Hook de TanStack Query para impresoras térmicas.

Exporta:	`useMostrarImpresorasXCajaQueryStack`
Dependencias:	`@tanstack/react-query, ImpresorasStore, crudImpresoras`
Lógica Clave:	Obtiene la configuración de la impresora asignada a la caja activa.

src/tanstack/LoginStack.jsx

AUTENTICACIÓN Y CLAVES

Hook de TanStack Query para autenticación y claves.

Exporta:	`useIniciarSesionConEmailMutationStack, useMostrarContrase`
Dependencias:	`@tanstack/react-query, AuthContext, supabase.config`
Lógica Clave:	Gestiona el inicio de sesión con email y contraseña en Supabase Auth.

src/tanstack/MetodosPagoStack.jsx

MEDIOS DE PAGO

Hook de TanStack Query para medios de pago.

Exporta:	`useMostrarMetodosDePagoQueryStack, useInsertarMetodosPagoMutationStack`
Dependencias:	`@tanstack/react-query, MetodosPagoStore, crudMetodoPago`
Lógica Clave:	Carga los medios de cobro que se renderizan en el teclado numérico del POS.

src/tanstack/ModulosStack.jsx

MÓDULOS DE SISTEMA

Hook de TanStack Query para módulos de sistema.

Exporta:	`useMostrarModulosQueryStack`
Dependencias:	`@tanstack/react-query, ModulosStore, crudModulos`
Lógica Clave:	Provee la lista de módulos para la asignación de permisos.

src/tanstack/MonedaStack.jsx

SÍMBOLO Y FORMATO MONETARIO

Hook de TanStack Query para símbolo y formato monetario.

Exporta:	`useMostrarMonedaQueryStack`
Dependencias:	`@tanstack/react-query, MonedaStore, crudMoneda`
Lógica Clave:	Sincroniza la moneda activa de la empresa en toda la UI.

src/tanstack/MovimientosCajaStack.jsx

CONTROL DE EFECTIVO Y CIERRE DE TURNO

Hook de TanStack Query para control de efectivo y cierre de turno.

Exporta:	`useInsertarIngresosSalidasCajasMutationStack, useMostrarEfectivoSinVentasMovCajasQueryStack, useMostrarDetalleEfectivoSinVentasMovCajasQueryStack, useMostrarVentasMetodoPagoMovCajaQueryStack, useTerminarTurnoMutationStack, useMostrarMovimientosCajaXEmpresYFechaQueryStack`
Dependencias:	`@tanstack/react-query, MovCajaStore, CierreCajaStore, crudMovimientosCaja`
Lógica Clave:	Calcula el arqueo de caja final comparando ventas en efectivo, entradas y salidas manuales frente al conteo ciego.

src/tanstack/MovimientosStock.jsx

KARDEX DE INVENTARIO

Hook de TanStack Query para kardex de inventario.

Exporta:	`useMostrarMovimientosStockQueryStack, useInsertarMovStcoMutationStack`
Dependencias:	`@tanstack/react-query, MovStockStore, crudMovimientosStock`
Lógica Clave:	Registra los ingresos y egresos extraordinarios de mercancía.

src/tanstack/PermisosStack.jsx

SEGURIDAD Y RBAC

Hook de TanStack Query para seguridad y rbac.

Exporta:	`useMostrarPermisosGlobalesQueryStack, useMostrarPermisosConfiguracionesQueryStack, useMostrarPermisosPorUsuariosQueryStack`
Dependencias:	`@tanstack/react-query, PermisosStore, crudPermisos`
Lógica Clave:	Alimenta las compuertas 'ProtectedRoute' y 'UseValidarPermisosOpertivos'.

src/tanstack/RolesStack.jsx

ROLES DE PERSONAL

Hook de TanStack Query para roles de personal.

Exporta:	`useMostrarRolesQueryStack`
Dependencias:	`@tanstack/react-query, RolesStore, crudRol`
Lógica Clave:	Consulta los roles disponibles para asignar a empleados.

src/tanstack/SerealizacionStack.jsx

SERIES Y COMPROBANTES FISCALES

Hook de TanStack Query para series y comprobantes fiscales.

Exporta:	`useMostrarSerealizacionesQueryStack, useMostrarSerealizacionesVentasQueryStack, useEditarSerealizacionDefaultMutationStack`
Dependencias:	`@tanstack/react-query, SerealizacionesStore, crudSerealizaciones`
Lógica Clave:	Controla qué comprobante (Ticket, Factura) y serie se utilizará al cobrar.

`src/tanstack/UsuariosStack.jsx` PERSONAL Y PREFERENCIAS VISUALES

Hook de TanStack Query para personal y preferencias visuales.

Exporta:	`useMostrarUsuariosQueryStack, useInsertarUsuariosPorEmpresaMutationStack, useEditarUsuarioMutationStack, useEditarTemaMutationStack`
Dependencias:	`@tanstack/react-query, UsuariosStore, crudUsuarios`
Lógica Clave:	Permite crear cajeros, editar datos y guardar la preferencia del tema visual.

2.7 Capa de Vistas y Páginas de Navegación (/pages – 22 Páginas) Pages

Componentes montados directamente por el enrutador de React Router DOM. Inyectan stores, gestionan estados locales de vista y orquestan los templates correspondientes.

`src/pages/POS.jsx` TERMINAL PUNTO DE VENTA PRINCIPAL

Página de vista para terminal punto de venta principal.

Exporta:	`export function POS()`
Dependencias:	`POSTemplate, PantallaAperturaCaja, Spinner1, CierreCajaStore, MetodosPagoStack`
Lógica Clave:	Evalúa si el usuario activo tiene una caja abierta con 'dataCierreCaja'. Si no hay apertura registrada, bloquea la venta y monta "; si existe apertura, renderiza ".

`src/pages/Dashboard.jsx` PANEL EJECUTIVO Y MÉTRICAS ANALÍTICAS

Página de vista para panel ejecutivo y métricas analíticas.

Exporta:	`export function Dashboard()`
Dependencias:	`DashboardTemplate`
Lógica Clave:	Punto de entrada de analítica comercial: métricas de ventas en tiempo real, productos más vendidos, ganancias brutas y gráficos comparativos.

`src/pages/Inventario.jsx` GESTOR INTEGRAL DE INVENTARIO Y STOCK

Página de vista para gestor integral de inventario y stock.

Exporta:	`export function Inventario()`
Dependencias:	`InventarioTemplate, Buscador, TablaInventarios`
Lógica Clave:	Permite consultar existencias multialmacén, realizar ajustes manuales de stock y exportar listados de inventario.

src/pages/Creditos.jsx

CUENTAS POR COBRAR Y GESTIÓN DE CRÉDITOS

Página de vista para cuentas por cobrar y gestión de créditos.

Exporta:	`export function Creditos()`
Dependencias:	`CreditosTemplate`
Lógica Clave:	Visualiza las deudas pendientes de clientes, abonos acumulados y acceso a la recepción de pagos.

src/pages/MovimientosCaja.jsx

AUDITORÍA DE FLUJO DE EFECTIVO

Página de vista para auditoría de flujo de efectivo.

Exporta:	`export function MovimientosCaja()`
Dependencias:	`MovimientosCajaTemplate`
Lógica Clave:	Historial cronológico de todos los movimientos de caja: ingresos, egresos, ventas en efectivo y tarjetas.

src/pages/Categorias.jsx

ADMINISTRACIÓN DE FAMILIAS DE PRODUCTOS

Página de vista para administración de familias de productos.

Exporta:	`export function Categorias()`
Dependencias:	`CategoriasTemplate`
Lógica Clave:	Página de administración para clasificar productos por categorías con colores e iconos.

src/pages/Productos.jsx

CATÁLOGO MAESTRO DE PRODUCTOS

Página de vista para catálogo maestro de productos.

Exporta:	`export function Productos()`
Dependencias:	`ProductosTemplate`
Lógica Clave:	CRUD de productos: creación, edición de precios de compra/venta, código de barras, fotos y asignación de stock.

src/pages/Almacenes.jsx

GESTIÓN DE BODEGAS Y ALMACENES FÍSICOS

Página de vista para gestión de bodegas y almacenes físicos.

Exporta:	`export function Almacenes()`
Dependencias:	`AlmacenesTemplate`
Lógica Clave:	Configura las diferentes bodegas o depósitos de la empresa vinculados a sucursales.

src/pages/SucursalesCaja.jsx

SEDES Y CAJAS REGISTRADORAS

Página de vista para sedes y cajas registradoras.

Exporta:	`export function SucursalesCaja()`
Dependencias:	`SucursalesCajaTemplate`
Lógica Clave:	Estructura multi-sucursal y terminales POS habilitadas en cada sede.

src/pages/ClientesProveedores.jsx

DIRECTORIO DE TERCEROS

Página de vista para directorio de terceros.

Exporta:	`export function ClientesProveedores()`
Dependencias:	`ClientesProveedoresTemplate`
Lógica Clave:	Mantiene la base de datos de clientes comerciales y proveedores mayoristas.

src/pages/MetodosPago.jsx

CONFIGURACIÓN DE MEDIOS DE PAGO

Página de vista para configuración de medios de pago.

Exporta:	`export function MetodosPago()`
Dependencias:	`MetodosPagoTemplate`
Lógica Clave:	Permite habilitar o deshabilitar formas de cobro (Efectivo, Tarjeta, Transferencias, etc.).

src/pages/Empresa.jsx

PERFIL CORPORATIVO Y AJUSTES FISCALES

Página de vista para perfil corporativo y ajustes fiscales.

Exporta:	`export function Empresa()`
Dependencias:	`EmpresaTemplate, Outlet, NavLink`
Lógica Clave:	Gestión del nombre fiscal, NIT/RUT, logo corporativo, régimen tributario y subrutas de configuración.

src/pages/Configuraciones.jsx

CENTRO DE CONTROL DEL SISTEMA

Página de vista para centro de control del sistema.

Exporta:	`export function Configuraciones()`
Dependencias:	`ConfiguracionesTemplate`
Lógica Clave:	Menú centralizado de accesos rápidos a todos los módulos administrativos y de parametrización.

`src/pages/ConfiguracionTicket.jsx`

PERSONALIZADOR DE FORMATO DE TICKET

Página de vista para personalizador de formato de ticket.

Exporta:	`export function ConfiguracionTicket()`
Dependencias:	`useEmpresaStore, CreatePdf, styled-components`
Lógica Clave:	Editor visual con previsualización en vivo para ajustar encabezados, pies de página, notas y anchos del ticket térmico.

`src/pages/Impresoras.jsx`

ENLACE DE IMPRESORAS TÉRMICAS

Página de vista para enlace de impresoras térmicas.

Exporta:	`export function Impresoras()`
Dependencias:	`ImpresorasTemplate`
Lógica Clave:	Asigna qué impresora física del sistema operativo atenderá las órdenes de impresión de cada caja.

`src/pages/SerializacionComprobantes.jsx`

SERIES Y FOLIOS DE FACTURACIÓN

Página de vista para series y folios de facturación.

Exporta:	`export function SerializacionComprobantes()`
Dependencias:	`SerealizacionesTemplate`
Lógica Clave:	Configura los prefijos, números consecutivos y tipos de documentos tributarios autorizados.

`src/pages/Usuarios.jsx`

ADMINISTRACIÓN DE PERSONAL Y ROLES

Página de vista para administración de personal y roles.

Exporta:	`export function Usuarios()`
Dependencias:	`UsuariosTemplate`
Lógica Clave:	Gestión de empleados, asignación de credenciales, roles y matriz de permisos operativos.

`src/pages/Perfil.jsx`

PERFIL DEL USUARIO EN SESIÓN

Página de vista para perfil del usuario en sesión.

Exporta:	`export function Perfil()`
Dependencias:	`useUsuariosStore, useAuthStore, styled-components`
Lógica Clave:	Permite al usuario conectado cambiar su avatar, contraseña y preferencia del tema de interfaz.

`src/pages/AbrirCajaSerealizacion.jsx` WIZARD DE INICIALIZACIÓN DE CAJA

Página de vista para wizard de inicialización de caja.

Exporta:	`export function AbrirCajaSerealizacion()`
Dependencias:	`Wizard components, CajasStack`
Lógica Clave:	Asistente inicial que guía al cajero en la selección de sucursal, caja, serialización y fondo de caja.

`src/pages/SinPermiso.jsx` PANTALLA DE ACCESO DENEGADO (403)

Página de vista para pantalla de acceso denegado (403).

Exporta:	`export function SinPermiso()`
Dependencias:	`framer-motion, VolverBtn, styled-components`
Lógica Clave:	Vista de seguridad renderizada cuando un usuario sin privilegios intenta entrar a una ruta no autorizada.

`src/pages/Home.jsx` PÁGINA PRINCIPAL DE BIENVENIDA

Página de vista para página principal de bienvenida.

Exporta:	`export function Home()`
Dependencias:	`HomeTemplate, LandingPagesWelcome`
Lógica Clave:	Pantalla de bienvenida y resumen de inicio con accesos directos.

`src/pages/Login.jsx` PORTAL DE AUTENTICACIÓN TEMÁTICO

Página de vista para portal de autenticación temático.

Exporta:	`export function Login()`
Dependencias:	`LoginTemplate, EscenaHalloween`
Lógica Clave:	Formulario de login animado con fondo interactivo de Halloween y validación de credenciales.

2.8 Capa de Componentes UI: Átomos, Moléculas, Organismos y Templates

Atomic Design

Componentes Templates (19 Layouts Estructurales en src/components/templates/)

`src/components/templates/POSTemplate.jsx` LAYOUT MAESTRO DE LA PANTALLA DE VENTA POS

Área de detalle de venta a la izquierda, área de teclado/totales a la derecha, barra superior con selector de almacén y decoraciones temáticas vectoriales de Halloween (arañas y murciélagos).

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/DashboardTemplate.jsx`

ESTRUCTURA DE CUADRÍCULAS DEL DASHBOARD

Dispone las tarjetas de KPIs, gráficos de Recharts de ventas por mes, top productos y filtros de rango de fechas.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/CategoriasTemplate.jsx`

PLANTILLA DE GESTIÓN DE CATEGORÍAS

Distribuye el encabezado con botón de nueva categoría, buscador y la tabla interactiva de familias.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/ProductosTemplate.jsx`

PLANTILLA DEL CATÁLOGO DE PRODUCTOS

Organiza la barra de búsqueda, selector de almacén, botón de alta y la tabla general de inventario.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/ClientesProveedoresTemplate.jsx`

PLANTILLA DE TERCEROS

Soporta pestañas dinámicas para alternar entre la gestión de clientes y proveedores.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/CreditosTemplate.jsx`

PLANTILLA DE CUENTAS POR COBRAR

Estructura el resumen de cartera crediticia, buscador de deudores y tabla de créditos activos.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/MovimientosCajaTemplate.jsx` PLANTILLA DE MOVIMIENTOS DE CAJA

Layout con selector de fechas para auditar entradas, salidas y flujo de caja.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/AlmacenesTemplate.jsx` PLANTILLA DE GESTIÓN DE ALMACENES

Lista de almacenes por sucursal con tarjetas descriptivas y modales de edición.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/SucursalesCajaTemplate.jsx` PLANTILLA DE SEDES Y CAJAS

Estructura jerárquica para administrar sedes y sus cajas terminales asociadas.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/EmpresaTemplate.jsx` PLANTILLA DE AJUSTES CORPORATIVOS

Menú de navegación lateral interno para datos básicos, logo, monedas y configuración de tickets.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/ConfiguracionesTemplate.jsx` PLANTILLA DEL CENTRO DE CONTROL

Mosaico responsive de tarjetas con iconos para ingresar a cada módulo del sistema.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/ImpresorasTemplate.jsx` PLANTILLA DE IMPRESORAS TÉRMICAS

Visualización de impresoras detectadas por QZ-Tray y asignación por caja.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/UsuariosTemplate.jsx` PLANTILLA DE PERSONAL Y PERMISOS

Administración de usuarios con panel lateral de matriz de permisos dinámicos.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/MetodosPagoTemplate.jsx` PLANTILLA DE MÉTODOS DE PAGO

Cuadrícula de medios de cobro habilitados con botones de edición.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/SerealizacionesTemplate.jsx` PLANTILLA DE FACTURACIÓN Y SERIES

Gestión de series fiscales, tipos de documento y comprobante predeterminado.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/LoginTemplate.jsx` PLANTILLA DE INICIO DE SESIÓN

Diseño inmersivo con escena vectorial de Halloween y formulario de acceso.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/HomeTemplate.jsx` PLANTILLA DE INICIO

Layout general de bienvenida con accesos rápidos.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/CrudTemplate.jsx` PLANTILLA GENÉRICA DE OPERACIONES CRUD

Base reusable con título, botón de acción, buscador y contenedor de tabla.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

`src/components/templates/404.jsx` PLANTILLA DE ERROR 404 (NO ENCONTRADO)

Vista informativa cuando se solicita una ruta inexistente con botón de retorno.

Exporta:	`export const / export function`
Dependencias:	`styled-components, framer-motion, Atomic components`
Lógica Clave:	Define la geometría visual, breakpoints y acomodo de componentes hijos.

Organismos de Dominio (src/components/organismos/)

Componentes complejos agrupados por submódulo de negocio:

- **POSDesing (11 archivos):** `AreaDetalleventaPos.jsx` (grilla de productos agregados con ajuste de cantidades), `AreaTecladoPos.jsx` (teclado táctil de cobro y métodos de pago), `FooterPos.jsx` (accesos rápidos por teclado), `HeaderPos.jsx` (reloj, selector de almacén y usuario), `IngresoCobro.jsx` (cálculo de efectivo recibido y vuelto), `MenuFlotante.jsx` (acciones secundarias), `PanelBuscador.jsx` (búsqueda instantánea de productos), `PantallaCobro.jsx` (modal principal de liquidación de venta), `PantallaMuestraValoresVenta.jsx` (pantalla para el cliente), `SelectAlmacen.jsx` (cambio de almacén de despacho), `TotalPos.jsx` (gran total destacado en pantalla).
- **CajaDesing (6 archivos en POSDesing/CajaDesing/):** `PantallaAperturaCaja.jsx` (declaración de saldo inicial), `PantallaCierreCaja.jsx` (resumen y confirmación de cierre), `PantallaConteoCaja.jsx` (arqueo ciego de billetes y monedas), `PantallaIngresoSalidaDinero.jsx` (entradas y egresos extraordinarios de caja), `PantallaAbonoCredito.jsx` (recepción de pagos de clientes a crédito), `CardListCajas.jsx` (selector de terminal).
- **DashboardDesign (11 archivos):** `DashboardHeader.jsx` (filtros temporales), `CardTotales.jsx`, `CantidadVentas.jsx`, `SumarVentas.jsx`, `Ganancias.jsx`, `ChartVentas.jsx` (gráfico temporal de ingresos), `ChartProductosTop5.jsx` (gráfico de dona de artículos más vendidos), `CardMovimientosCajaLive.jsx`, `CardMovimientosProductosTopMonto.jsx`, `DateRangeFilter.jsx`, `DateRangeFilterMovCajas.jsx`.
- **Formularios Modales (14 archivos en formularios/):** Formularios controlados con validación para `RegistrarProductos.jsx`, `RegistrarCategorias.jsx`, `RegistrarClientesProveedores.jsx`, `RegistrarAlmacen.jsx`, `RegistrarCaja.jsx`, `RegistrarSucursal.jsx`, `RegistrarUsuarios.jsx`, `RegistrarCreditos.jsx`, `RegistrarMetodosPago.jsx`, `RegistrarSerializacion.jsx`, `RegistrarInventario.jsx`, `RegistrarmovimientocreditoVenta.jsx`, complementados por `InputText.jsx` e `InputText2.jsx`.
- **Tablas de Datos (14 archivos en tablas/):** Implementaciones con TanStack Table v8: `TablaProductos.jsx`, `TablaCategorias.jsx`, `TablaClientesProveedores.jsx`, `TablaInventarios.jsx`, `TablaCreditos.jsx`, `TablaMetodosPago.jsx`, `TablaMovimientosCaja.jsx`, `TablaMovimientosCajaLive.jsx`, `TablaProductosTop10.jsx`,

`TablaSerealizaciones.jsx`, `TablaUsuarios.jsx`, más utilidades de control: `AccionesTabla.jsx`, `ContentAccionesTabla.jsx`, `Paginacion.jsx`.

- **Sidebar & Navegación (2 archivos):** `Sidebar.jsx` (barra lateral expandible con enlaces animados y logotipo) y `MenuMovil.jsx` (menú responsive estilo drawer para dispositivos móviles).
- **Otros Organismos:** `Buscador.jsx`, `Reloj.jsx`, `ToggleTema.jsx` (conmutador Light/Dark animado), `Checkbox1.jsx`, `Switch1.jsx`, `Selector.jsx`, `ListaDesplegable.jsx`, `Footer.jsx`, `BasicosConfig.jsx`, `MonedaConfig.jsx`, `HeaderImpresoras.jsx`, `PermisosUser.jsx`, `CardModos.jsx`, `EscenaHalloween.jsx`, `LandingPagesWelcome.jsx`, `CardFuncion.jsx`, `ListAlmacenes.jsx`, `ListSucursales.jsx`.

Átomos y Moléculas (src/components/atomos/ y src/components/moleculas/)

- **Átomos (6 archivos):** `AbrirCajaImpresora.jsx` (conector QZ con pulsos ESC/POS hexadecimales para cajón), `ContainerSelector.jsx`, `Icono.jsx`, `Linea.jsx` (separador estético), `Lottieanimation.jsx` (renderizador de animaciones JSON), `Titles.jsx` (títulos estandarizados).
- **Moléculas (13 archivos):** `Btn1.jsx` (botón primario con soporte de iconos), `BtnLink.jsx`, `Btngenerarcodigo.jsx`, `HistorialCreditoModal.jsx`, `ImageContent.jsx`, `IndicadorConexion.jsx`, `LiveIndicator.jsx`, `Spinner1.jsx` (animación de carga principal temática), `SpinnerSecundario.jsx`, `TicketModal.jsx` (visor de ticket de venta), `TicketModalCredito.jsx` (visor de recibo de crédito), `Toogle.jsx`, `VolverBtn.jsx`.
- **Componentes UI (7 archivos en src/components/ui/):** `AnimatedGrid.jsx`, `BtnClose.jsx`, `ButtonDashed.jsx`, `Buscador.jsx`, `SelectList.jsx`, `Check.jsx`, `RadioChecks.jsx`, `Switch.jsx`.

2.9 Reportes de Impresión Térmica, Utilidades y Sistema de Diseño

Reports & Utils

`src/reports/TicketVenta.jsx`

GENERADOR DE TICKET DE VENTA TÉRMICO

Genera la estructura de documento para pdfmake adaptada al ancho térmico estándar (204.09 pt / 80mm o 58mm). Dibuja líneas punteadas con canvas vectorial, renderiza logo corporativo en Base64, lista productos con cantidades y precios, desglosa formas de pago y convierte el monto total a texto en español usando 'written-number'.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

`src/reports/TicketAbonoCredito.jsx`

COMPROBANTE DE RECIBO DE CAJA POR ABONO

Genera el recibo fiscal cuando un cliente abona dinero a una cuenta de crédito pendiente, detallando el saldo anterior, el monto entregado y el nuevo saldo insoluto.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/reports/TicketEntradasSalidas.jsx

COMPROBANTE DE MOVIMIENTO DE CAJA

Documento que certifica la extracción o inyección manual de efectivo en la gaveta, detallando el usuario responsable, motivo y fecha.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/reports/TicketPrueba.jsx

TICKET DE PRUEBA DE CONECTIVIDAD

Imprime un comprobante de diagnóstico para verificar la alineación del papel, cabezal térmico y corte automático.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/utils/qzSecurity.js

CRIPTOGRAFÍA Y FIRMA RSA PARA QZ-TRAY

Contiene un certificado X.509 y clave privada RSA codificadas en Base64 embebidas. Expone 'configurarFirmaQZ()' que intercepta las peticiones de QZ-Tray y las firma criptográficamente con el algoritmo 'SHA512withRSA' usando la librería 'jsrsasign' para autorizar el acceso directo al hardware sin diálogos de advertencia del navegador.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/utils/CreatePdf.jsx

MOTOR DE DESPACHO DE COMPROBANTES PDF

Envuelve la API de 'pdfmake' y 'print-js'. Soporta dos modos de salida: 'b64' (retorna la cadena Base64 del documento para almacenamiento o envío por email) y 'print' (despacha el PDF directamente al spooler de impresión del sistema operativo mediante print-js).

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/utils/RobotoFonts.js

FUENTES TIPOGRÁFICAS EMBEBIDAS EN BASE64

Archivo de datos con el binario completo de la familia tipográfica 'Roboto' (Regular, Medium, Italic) codificado en cadenas Base64. Permite a pdfmake renderizar caracteres especiales y acentos en español sin depender de conexiones a internet.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/utils/Conversiones.jsx

FORMATEADORES DE MONEDA Y CADENAS

Funciones utilitarias: 'FormatearNumeroDinero(monto, currency, iso)' (formatea con Intl a la moneda de la empresa), 'FormatearNumeroDineroSinIsoYCurrency', 'ConvertirCapitalize', y 'urlToBase64' para transformar logos web en imágenes imprimibles.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/utils/dataEstatica.jsx

CONSTANTES Y ENLACES DE NAVEGACIÓN

Declara la estructura de los menús 'LinksArray' (Inicio, Dashboard, Vender, Inventario), 'SecondarylinksArray' (Configuración, Mi Perfil) y 'DesplegableUser'.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/utils/animations.js

VARIANTES DE ANIMACIÓN FRAMER MOTION

Colección de objetos de configuración para animaciones de entrada/salida (fades, scales, slides, spring physics) compartidos en toda la aplicación.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/styles/themes.jsx

PALETAS TEMÁTICAS LIGHT / DARK / HALLOWEEN

Define los temas visuales con tokens hexadecimales: fondo general ('bgtotal'), acentos temáticos ('halloweenPrimary: #D97706', 'halloweenSecondary: #7C3AED', 'halloweenAccent: #F97316'), sombras violetas y bordes brillantes.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/styles/GlobalStyles.jsx

ESTILOS GLOBALES Y NORMALIZACIÓN CSS

Aplica reset CSS universal, reglas de tipografía, estilos de barra de desplazamiento personalizada (scrollbar) y selección de texto.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/styles/breakpoints.jsx

DEFINICIÓN DE BREAKPOINTS RESPONSIVE

Exporta el objeto 'Device' con media queries estándar (mobile, tablet, desktop, desktopL) para Styled-Components.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/styles/Keyframes.jsx

ANIMACIONES CSS NATIVAS

Keyframes declarados con 'keyframes' de Styled-Components: pulsaciones, flotación de murciélagos, giros de loader y destellos.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

src/styles/variables.jsx

ICONOS VECTORIALES Y VARIABLES CENTRALIZADAS

Objeto centralizado 'v' que asocia iconos SVG de React Icons (FaCheck, MdClose, BiSearch, etc.) con identificadores reutilizables.

Exporta:	`Funciones de utilidad, generadores de reportes y tokens de diseño.`
Dependencias:	`pdfmake, print-js, jsrsasign, qz-tray, styled-components`
Lógica Clave:	Módulos auxiliares de alto impacto en la experiencia de impresión y diseño visual.

2.10 Recursos Multimedia y Animaciones Vectoriales (/src/assets) Assets

El directorio `src/assets/` alberga archivos visuales y vectores Lottie para la ambientación tematica de Halloween:

- `halloween pumpkin.json`: Animación interactiva Lottie de una calabaza sonriente utilizada en modales de éxito.
- `Frankenstein.json`: Animación vectorial de personaje monstruo para estados de espera y carga en segundo plano.
- `Handchi spooky.json`: Vector animado de mano fantasmagórica en la pantalla de bienvenida.
- `add to cart.json`: Animación de micro-interacción al añadir artículos al carrito de compra.
- `vacio2.json` y `vacioanimation.json.json`: Vectores Lottie para pantallas de listas vacías (sin productos, sin ventas registradas).
- `Cobrar.png` y `react.svg`: Gráficos de alta resolución para paneles de teclado táctil y branding.

CAPÍTULO 3

Guía para Programadores: Código Clave y Cómo se Usa

Manual de referencia técnica con fragmentos reales del código fuente del proyecto. Explica la lógica interna, el flujo de llamadas entre capas, la orquestación de hardware y las directrices paso a paso para extender o mantener la plataforma.

3.1 Flujo de Autenticación, Sesión Diferida y Verificación Dinámica RBAC

El sistema implementa un ciclo de autenticación blindado contra carreras de condiciones (race conditions) en el navegador. La recomendación oficial de Supabase prohíbe realizar consultas asíncronas dentro del callback de `onAuthStateChange` porque puede causar bloqueos del hilo de eventos. POSHalloween resuelve esto difiriendo la inicialización de la empresa en la cola de microtareas:

src/context/AuthContext.jsx • Listener de Sesión Desacoplado JAVASCRIPT / REACT

```jsx
export const AuthContextProvider = ({ children }) => {
  const { insertarEmpresa } = useEmpresaStore();
  const [user, setUser] = useState(undefined);
  const [isCheckingAuth, setIsCheckingAuth] = useState(true);

  useEffect(() => {
    const { data } = supabase.auth.onAuthStateChange((value, session) => {
      const usuarioSesion = session?.user ?? null;
      setUser(usuarioSesion);
      setIsCheckingAuth(false);

      if (usuarioSesion) {
        // IMPORTANTE: Se difiere la llamada con setTimeout( ... , 0) para no ejecutar
        // consultas pesadas de Supabase dentro del callback síncrono del evento de Auth.
        setTimeout(() => {
          insertarDatos(usuarioSesion.id, usuarioSesion.email).catch((error) =>
            console.error(error)
          );
        }, 0);
      }
    });

    return () => data.subscription.unsubscribe();
  }, []);

  const insertarDatos = async (id_auth, correo) => {
    const response = await MostrarUsuarios({ id_auth });
    if (!response) {
      // Si el usuario ingresa por primera vez, crea automáticamente el registro de su empresa
      await insertarEmpresa({ id_auth, correo });
    }
  };

  return (
    <AuthContext.Provider value={{ user, isCheckingAuth }}>
      {children}
    </AuthContext.Provider>
  );
};
```

En la capa de enrutamiento, el componente `ProtectedRoute.jsx` actúa como policía de fronteras. No se basa en un booleano genérico 'esAdmin', sino que verifica dinámicamente si la URL actual existe en la colección de permisos del usuario:

src/hooks/ProtectedRoute.jsx • Validación Dinámica de Rutas RBAC — JAVASCRIPT / REACT

```jsx
export const ProtectedRoute = ({ children, accesby }) => {
  const { user } = userAuth();
  const location = useLocation();
  const { datausuarios } = useUsuariosStore();
  const { data: dataPermisosGlobales, isLoading: isLoadingPermisos } =
    useMostrarPermisosGlobalesQueryStack();

  const isLoading = !datausuarios || isLoadingPermisos;
  if (user === undefined) return <Spinner1 />;

  if (accesby === "authenticated") {
    if (!user) return <Navigate to="/login" />;
    if (isLoading) return <SpinnerSecundario texto="🔐 Verificando permisos" />;

    // Valida contra la base de datos si la URL actual coincide con un módulo permitido
    const hasPermission = dataPermisosGlobales?.some((item) => {
      return item.modulos?.link === location.pathname;
    });

    if (!hasPermission) {
      // Si no tiene permiso, lo desvía a la pantalla 403 sin romper la aplicación
      return <Navigate to="/sin-permiso" />;
    }
    return children;
  }
};
```

3.2 El Ciclo de Vida de una Venta en el POS (Carrito, Zustand y Mutación Atómica)

El carrito de ventas no reside en el servidor para evitar latencia innecesaria de red. Se administra localmente en memoria y en `localStorage` mediante `useCartVentasStoreTemporal`. Cuando el cajero añade productos, el store calcula subtotales en tiempo real:

src/store/CartVentasStoreTemporal.jsx • Lógica Reactiva del Carrito JAVASCRIPT / ZUSTAND

```javascript
function calcularTotal(items) {
  return items.reduce(
    (total, item) => total + item._precio_venta * item._cantidad,
    0
  );
}

export const useCartVentasStoreTemporal = create(
  persist(
    (set) => ({
      items: [],
      total: 0,
      statePantallaCobro: false,

      addItem: (p) =>
        set((state) => {
          const existingItem = state.items.find(
            (item) => item._id_producto === p._id_producto
          );
          if (existingItem) {
            // Si el artículo ya está en el carrito, incrementa la cantidad
            const updatedItems = state.items.map((item) => {
              if (item._id_producto === p._id_producto) {
                const newQuantity = item._cantidad + (p._cantidad || 1);
                return {
                  ...item,
                  _cantidad: newQuantity,
                  _total: newQuantity * item._precio_venta,
                };
              }
              return item;
            });
            return { items: updatedItems, total: calcularTotal(updatedItems) };
          } else {
            // Si es un producto nuevo, lo anexa al array
            return {
              items: [...state.items, p],
              total: calcularTotal([...state.items, p]),
            };
          }
        }),
    }),
    { name: "cart-ventas-pos" }
  )
);
```

El clímax de la venta ocurre en `VentasStack.jsx` dentro del hook `useConfirmarVentasMutationStack`. Esta mutación orquesta múltiples operaciones críticas en secuencia estricta:

src/tanstack/VentasStack.jsx • Confirmación Atómica de la Venta JAVASCRIPT / TANSTACK QUERY

```javascript
export const useConfirmarVentasMutationStack = ({
  imprimirDirectoTicket,
  imprimirConVentanaEmergente,
}) => {
  // Extrae parámetros de los stores de Zustand
  const { restante, idventa, vuelto, confirmarVenta, valoresPago, resetState } = useVentasStore();
  const { datausuarios } = useUsuariosStore();
  const { itemSelectComprobanteSelect } = useSerealizacionesStore();
  const { dataCierreCaja } = useCierreCajaStore();
  const { total } = useDetalleVentasStore();
  const { mostrarAlertasStockXVenta } = useStockStore();

  async function ConfirmarVenta() {
    // 1. Regla de negocio: El restante por pagar debe ser exactamente 0
    if (restante !== 0) {
      toast.warning("Falta completar el pago, el restante tiene que ser cero");
      return;
    }

    // 2. Ensamblar parámetros para la función RPC de PostgreSQL
    const pventas = {
      _id_venta: idventa,
      _id_usuario: datausuarios?.id,
      _vuelto: vuelto,
      _id_tipo_comprobante: itemSelectComprobanteSelect?.id_tipo_comprobante,
      _serie: itemSelectComprobanteSelect?.serie,
      _id_sucursal: dataCierreCaja?.caja?.id_sucursal,
      _fecha: fechaActual,
      _monto_total: total,
    };

    // 3. Invocar procedimiento almacenado en Supabase
    const responseVenta = await confirmarVenta(pventas);

    // 4. Desglosar pagos en movimientos_caja para cuadre de caja (Efectivo / Tarjeta / etc.)
    for (const [tipo, monto] of Object.entries(valoresPago)) {
      if (monto > 0) {
        const montoNetoCaja = tipo === "Efectivo" ? monto - vuelto : monto;
        await insertarMovcaja({
          fecha_movimiento: fechaActual,
          tipo_movimiento: "venta",
          monto: montoNetoCaja,
          id_cierre_caja: dataCierreCaja?.id,
          id_venta: idventa,
        });
      }
    }

    // 5. Disparar apertura del cajón monedero vía hardware
    try {
      await abrirCaja();
    } catch (e) {
      console.error("Error al abrir cajón:", e);
    }

    // 6. Impresión directa QZ o diálogo en pantalla
    dataImpresorasXCaja?.state
      ? await imprimirDirectoTicket()
      : await imprimirConVentanaEmergente(responseVenta);
```

```
    // 7. Auditar stock mínimo y alertar al cajero con SweetAlert2 si se agotó mercancía
    const alertas = await mostrarAlertasStockXVenta({ _id_venta: idventa });
    if (alertas?.length > 0) {
      Swal.fire({
        title: "¡Alerta de Stock Mínimo!",
        html: alertas.map(a => `<li>${a.nombre_producto}: ${a.stock_actual}</li>`).join(""),
        icon: "warning"
      });
    }
  }

  return useMutation({
    mutationKey: ["confirmar ventas"],
    mutationFn: ConfirmarVenta
  });
};
```

3.3 Orquestación de Hardware Térmico y Apertura de Cajón con QZ-Tray

Uno de los mayores desafíos en las aplicaciones web de punto de venta es comunicarse con impresoras térmicas y gavetas de dinero sin que el navegador bloquee la acción por políticas de seguridad de sandbox. POSHalloween soluciona esto integrando **QZ-Tray** con una conexión WebSocket asegurada mediante firma RSA:

src/utils/qzSecurity.js • Firma Digital RSA SHA-512 JAVASCRIPT / CRYPTO

```
import { KJUR, KEYUTIL, hextob64 } from "jsrsasign";
import qz from "qz-tray";

export function configurarFirmaQZ() {
  // Inyecta el certificado público de SoftCreate POS
  qz.security.setCertificatePromise((resolve) => resolve(CERT));
  qz.security.setSignatureAlgorithm("SHA512");

  // Firma criptográfica de cada mensaje enviado a QZ-Tray
  qz.security.setSignaturePromise((toSign) => (resolve, reject) => {
    try {
      const key = KEYUTIL.getKey(PRIVATE_KEY);
      const sig = new KJUR.crypto.Signature({ alg: "SHA512withRSA" });
      sig.init(key);
      sig.updateString(toSign);
      resolve(hextob64(sig.sign()));
    } catch (e) {
      reject(e);
    }
  });
}
```

Para expulsar el cajón monedero, `AbrirCajaImpresora.jsx` envía una secuencia de comandos hexadecimales ESC/POS que disparan una corriente eléctrica sobre los pines 2 y 5 del conector RJ11:

src/components/atomos/AbrirCajaImpresora.jsx • Pulsos Eléctricos Hexadecimales JAVASCRIPT / ESC-POS

```javascript
// Pulsos ESC/POS: pin 2 (1B700019FA) y pin 5 (1B700119FA)
const PULSOS = "1B700019FA" + "1B700119FA";

export async function abrirCaja() {
  if (!firmaConfigurada) {
    configurarFirmaQZ();
    firmaConfigurada = true;
  }
  if (!qz.websocket.isActive()) await qz.websocket.connect();

  // Localiza la impresora térmica mediante expresión regular en el spooler del OS
  const lista = await qz.printers.find();
  const termica = lista.find((n) => /pos|thermal|termic|80/i.test(n))
              || await qz.printers.getDefault();

  // Envío del comando raw en modo flavor hex
  await qz.print(qz.configs.create(termica), [
    { type: "raw", format: "command", flavor: "hex", data: PULSOS }
  ]);
}
```

3.4 Motor de Renderizado de Tickets Térmicos en PDF con pdfmake

A diferencia de las librerías que imprimen mediante captura de pantalla (HTML to Canvas), POSHalloween genera un documento vectorial genuino con `pdfmake`. Esto asegura que el texto sea perfectamente nítido en el cabezal térmico de 203 DPI:

src/reports/TicketVenta.jsx • Definición del Documento Térmico JAVASCRIPT / PDFMAKE

```javascript
// Ancho estándar de rollo de 80mm expresado en puntos tipográficos (72 pt = 1 pulgada)
const ANCHO_PAGINA = 204.09;
const MARGEN = 4;
const ANCHO_UTIL = ANCHO_PAGINA - MARGEN * 2;

// Dibuja una línea punteada decorativa mediante el API Canvas de pdfmake
const lineaPunteada = (margin = [0, 6, 0, 6]) => ({
  canvas: [
    {
      type: "line",
      x1: 0, y1: 0,
      x2: ANCHO_UTIL, y2: 0,
      lineWidth: 1,
      lineColor: "#000000",
      dash: { length: 2, space: 2 },
    },
  ],
  margin,
});
```

3.5 Control de Arqueo de Caja (Apertura Obligatoria, Conteo Ciego y Cierre)

Para erradicar faltantes de dinero y fraudes, la terminal no permite vender si no existe una apertura activa. En el cierre de turno, el cajero es sometido a un **conteo ciego**: debe contar y digitar billetes y monedas sin saber cuánto espera

el sistema. Al pulsar 'Cerrar Caja', la base de datos compara el conteo físico contra los movimientos registrados y emite el saldo diferencial.

3.6 Trazabilidad Multialmacén y Sistema de Alertas por Stock Mínimo

Cada producto puede existir en múltiples almacenes simultáneamente. El selector de almacén superior en el POS determina de qué bodega física se rebajarán las unidades vendidas. Si el producto se agota en el almacén de despacho principal, el modal `SelectAlmacen.jsx` permite seleccionar una bodega secundaria para transferir o despachar en el acto.

3.7 Guía Paso a Paso para Desarrolladores: Cómo Añadir un Nuevo Módulo al Sistema

Si un nuevo desarrollador necesita implementar una nueva entidad en el sistema (por ejemplo, *"Gestión de Devoluciones y Mermas"*), debe seguir rigurosamente este procedimiento de 5 etapas para conservar la coherencia arquitectónica:

1. **Etapa 1: Capa de Datos Supabase (`src/supabase/crudMermas.jsx`)**
   Crea las funciones exportadas `InsertarMermas`, `MostrarMermas` y `EliminarMermas` invocando los RPCs de Supabase correspondientes.
2. **Etapa 2: Capa de Estado Zustand (`src/store/MermasStore.jsx`)**
   Crea el hook `useMermasStore = create((set) => ({ ... }))` para gestionar el elemento seleccionado, estado de modales y filtros de búsqueda.
3. **Etapa 3: Capa de Sincronización TanStack Query (`src/tanstack/MermasStack.jsx`)**
   Declara `useMostrarMermasQueryStack()` y la mutación `useInsertarMermasMutationStack()` con su respectiva invalidación de claves de consulta (`queryClient.invalidateQueries(...)`).
4. **Etapa 4: Capa Visual Atómica (`src/components/organismos/formularios/` y `tablas/`)**
   Crea el formulario `RegistrarMermas.jsx` y la tabla `TablaMermas.jsx` utilizando los componentes atómicos existentes (`InputText`, `Btn1`, `Titles`).
5. **Etapa 5: Template, Página y Ruta Protegida**
   Crea `src/components/templates/MermasTemplate.jsx`, `src/pages/Mermas.jsx`, expórtalos en `src/index.js` y registra la ruta protegida en `src/routers/routes.jsx` con `accesby="authenticated"`. Finalmente, añade el módulo a la tabla `modulos` de la base de datos para que el módulo de permisos lo gobierne automáticamente.

CAPÍTULO 4

Resumen General, Conclusiones y Recomendaciones

Evaluación diagnóstica de la arquitectura de software de POS Halloween, análisis de calidad de código, detección de áreas de mejora técnica, matriz de despliegue en la nube y recomendaciones de gobernanza para el equipo.

4.1 Diagnóstico de Madurez y Fortalezas de la Arquitectura

El análisis exhaustivo del código fuente revela una solución de software moderna, estructurada bajo estándares de ingeniería avanzados para el desarrollo de aplicaciones de comercio y facturación. Entre sus principales fortalezas destacan:

- **Desacoplamiento Estricto en 3 Capas:** La separación entre componentes de presentación, orquestación con TanStack Query y acceso a base de datos con Supabase RPC previene la contaminación de la interfaz con lógica de red y facilita pruebas y mantenimiento independiente.
- **Gestión de Estado Híbrida Eficiente:** La combinación de *Zustand* para estados síncronos volátiles (como el carrito de compras y la apertura de modales) y *TanStack React Query* para el estado del servidor asíncrono con caché e invalidación reactiva asegura un rendimiento impecable de 60 cuadros por segundo en pantalla.
- **Integración Nítida con Hardware Físico:** La implementación de QZ-Tray con firmas digitales RSA y comandos ESC/POS puros dota a la aplicación web de capacidades equivalentes a un software de escritorio nativo (C# / Java), permitiendo disparar cajones monedero y cabezales térmicos sin interrupciones.
- **Seguridad RBAC Basada en Base de Datos:** El control de acceso dinámico mapeado contra la tabla `modulos` otorga una flexibilidad total al administrador para habilitar o vetar pantallas a nivel de rol o usuario sin necesidad de recompilar la aplicación.
- **Identidad Visual Inmersiva:** El sistema de diseño bajo Styled-Components con tokens Halloween (naranja calabaza, morado profundo y animaciones vectoriales) dota al producto de una personalidad comercial única sin sacrificar la ergonomía operativa.

4.2 Oportunidades Críticas de Refactorización y Mejora Técnica

Para elevar la madurez del proyecto hacia un estándar de nivel enterprise internacional, se recomiendan las siguientes acciones correctivas:

⚠ **1. Desacoplamiento de Clave Privada en qzSecurity.js**

El archivo `src/utils/qzSecurity.js` contiene una clave privada RSA codificada en Base64 en el bundle del cliente. Aunque QZ-Tray corre en la máquina local del cliente, exponer claves privadas en código JavaScript de frontend permite que cualquier usuario inspeccione la clave. Se recomienda trasladar el servicio de firma a una Supabase Edge Function que reciba el hash a firmar y retorne la cadena firmada, manteniendo la clave privada estrictamente protegida en las variables secretas de backend.

> ⚠️ **2. Normalización de Nombres de Archivo con Caracteres Especiales**
>
> Los archivos `src/store/ContraseaStore.jsx` y `src/supabase/crudContrasea.jsx` contienen caracteres con tildes o 'ñ' que en ciertos entornos operativos basados en Linux (como servidores de CI/CD de GitHub Actions o Docker) pueden originar fallos de codificación ('encoding mismatch'). Se aconseja renombrarlos a la convención estándar en inglés o español neutro: `ContrasenaStore.jsx` y `crudContrasena.jsx`.

> 💡 **3. Adopción Progresiva de TypeScript**
>
> Dado que el backend de Supabase y la Edge Function de facturas ya utilizan TypeScript, migrar gradualmente los archivos `.jsx` a `.tsx` permitirá aprovechar la generación automática de tipos de Supabase (`supabase gen types typescript`), garantizando autocompletado y validación estática de tipos en todos los RPCs y parámetros de venta.

4.3 Flujo de Despliegue en Producción (CI/CD, Firebase, Supabase)

Para publicar nuevas versiones del sistema en producción sin interrumpir la operación de los puntos de venta físicos, el equipo debe seguir este ciclo de despliegue:

Flujo de Compilación y Publicación — BASH / TERMINAL

```
# 1. Auditoría de calidad de código con ESLint
npm run lint

# 2. Generación del bundle optimizado para producción en carpeta dist/
npm run build

# 3. Despliegue del Frontend Single Page Application en Firebase Hosting
firebase deploy --only hosting

# 4. Despliegue de la función serverless de facturación en Supabase Edge Functions
supabase functions deploy enviar-facturas --project-ref [TU_PROJECT_REF]
```

4.4 Resumen de Convenciones y Recomendaciones de Mantenimiento

Área	Convención Obligatoria	Objetivo Arquitectónico
Barrel Exports	Todo nuevo componente, hook o store debe ser expuesto en `src/index.js`.	Mantener importaciones limpias de primer nivel y evitar refactorizaciones traumáticas.
Mutaciones	Toda mutación en TanStack Query debe ejecutar `queryClient.invalidateQueries(...)` en el handler `onSuccess`.	Evitar datos desactualizados en tablas y asegurar consistencia visual inmediata.
Manejo de Errores	Los errores en llamadas CRUD deben lanzarse con `throw new Error(...)` y capturarse en TanStack Query vía `toast.error()` o SweetAlert2.	Prevenir pantallas en blanco y proveer feedback descriptivo al cajero ante caídas de red.
Hardware Térmico	Las rutinas de impresión deben verificar previamente `dataImpresorasXCaja?.state` antes de disparar el spooler de QZ.	Permitir a cajas sin impresora física seguir vendiendo mediante diálogo PDF estándar.

Conclusión de la Auditoría Técnica

El proyecto **POS Halloween (SoftCreate POS)** representa una arquitectura robusta, bien modularizada y altamente especializada en el comercio minorista. Siguiendo las directrices documentadas en este manual, el equipo de ingeniería podrá escalar nuevas funcionalidades, mantener el rendimiento óptimo y garantizar la continuidad operativa de los puntos de venta.